

Prosjektrapport

ITF20319-1 24H - Software Engineering og testing

Arshad Abba
Andreas Isaksen
Dana Diyar Hafzullah
Kjell-Magne Larsen
Marius Rørmark
Michael Tarvid

22. november 2024
Halden



Innhold

1	Innledning	1
1.1	Om oppgaven	1
1.2	Begrensninger	1
1.3	Beskrivelse av klient	1
1.4	Beskrivelse av produktet	2
2	Brukerne	3
2.1	Personas	3
2.1.1	Sarah	3
2.1.2	Alex	4
2.1.3	Priya	4
2.2	Scenario	5
2.2.1	Sarah	5
2.2.1.1	Scenario 1	5
2.2.1.2	Scenario 2	6
2.2.2	Alex	6
2.2.2.1	Scenario 1	6
2.2.2.2	Scenario 2	7
2.2.2.3	Scenario 3	8
2.2.3	Priya	9
2.2.3.1	Scenario 1	9
2.2.3.2	Scenario 2	10
2.3	Brukerhistorier	10
2.3.1	Sarah	10
2.3.2	Alex	11

2.3.3	Priya	12
3	Kravspesifikasjon	13
3.1	Planning poker	13
3.2	Features	14
3.3	Krav	15
3.3.1	Funksjonelle krav	15
3.3.2	Tekniske krav	16
3.3.3	Designmessige krav	16
4	Utviklingsprosessen	18
4.1	Prosjektstyring	18
4.1.1	Prosjektstyringsverktøy - Notion	18
4.1.2	SCRUM	20
4.1.3	Rollefordeling	21
4.2	Timelister	22
4.3	Versjonskontroll	22
4.3.1	Github/Gitkraken	22
4.3.2	Workflow i Git	22
4.4	Sprinter	23
4.4.1	Sprint1 - retrospekt	23
4.4.1.1	Burndown chart	23
4.4.1.2	Ferdige features	24
4.4.2	Sprint 2 - retrospekt	24
4.4.2.1	Database	24
4.4.2.2	Nytt gruppemedlem	25
4.4.2.3	Burndown chart	26

4.4.3	Sprint 3 - retrospekt	26
4.5	Diskusjon rundt erfaringer	27
5	Løsning	28
5.1	Github repo	28
5.2	Avhengigheter	28
5.2.1	Java 21	28
5.2.2	JUnit 5 og Mockito	28
5.2.2.1	JSONassert/Unirest	28
5.2.3	Jackson	28
5.2.4	Javalin	29
5.2.5	MySQL JDBC driver	29
5.2.6	Azure MySQL server	29
5.2.7	AWS S3 bucket	29
5.2.8	JBcrypt	30
5.3	Klasser	30
5.4	Database	30
5.5	Kobling mot database	30
5.5.1	Mysql util	30
5.5.2	SQL spørringene	31
5.6	API endepunkt	31
5.7	Webløsning	31
5.7.1	Produktkatalog	31
5.7.1.1	Fremtidige Utviklingsprioriteringer	32
5.7.2	Artikler	33
5.7.3	Login funksjonalitet	35
5.7.4	Legge til ny bruker	36

5.7.5	Hvordan IOT enheter blir hentet ut	36
5.7.6	Personlig side etter login	36
6	Testing	37
6.1	CI Integrasjon	37
6.2	Integrasjonstest	37
6.3	Enhetstest	44
6.4	testStepCounterInitialCount enhetstest	45
6.5	testStepCounterIncrements enhetstest	45
6.6	testMultipleStepIncrements enhetstest	46
6.7	Resultat av tre enhetstester	46
7	Modeller	47
7.1	Mockups	47
7.1.1	Kompetanasetest	47
7.1.2	Konfigurerte enheter	48
7.1.3	Installasjonsguide - enkel	49
7.1.4	Installasjonsguide - Avansert	50
7.1.5	Produkt artikler, varslinger, annet	51
7.1.6	Forside kontrollpanel	52
7.1.7	IOT enhet - smart lås	53
7.2	Klassediagram	54
7.3	EER-diagram	55
7.4	Arkitektur	55
7.5	Login flytskjema	57
8	Drøfting	59
8.1	Krav til prototypen	59

8.1.1	Tillatelser	59
8.1.1.1	Kamera	59
8.1.1.2	Kalender	60
8.1.2	Gjenkjenn Enhet	60
8.1.3	Varsel/ping funksjoner	60
8.1.4	Produkt scan	60
8.1.5	Justering av varsler	61
8.1.6	Applikasjonens systemsikkerhet	62
8.1.6.1	Hashing av passord	62
8.1.6.2	Cookies	62
8.1.7	Målsetting for strømforbruk	63
8.1.8	Monitorering av strømforbruk	63
8.1.9	Status monitorering	64
8.1.10	Enhetsstyring	64
8.1.11	Enhets-API	65
8.1.12	Installasjonsprotokoll	65
8.1.13	Avansert/Enkel brukergrensesnitt for applikasjonen	65
8.1.14	Mine produkter - Meny	66
8.1.15	Design og universell utforming	66
8.1.16	Implementasjon av brukergrensesnitt	67
8.1.17	Kompetansetest	68
8.1.17.1	Forslag til spørsmål	68
8.1.18	Analyse av kompetansenivå	70
8.2	Bruk av rammeverk	70
9	Instruksjon for å kjøre programmet	71
9.1	Miljøoppsett	71

9.2	Kjøre tester	71
9.3	Kjøre program - GUI	72

Figurer

2.1	Personas - Sarah	3
2.2	Personas - Alex	4
2.3	Personas - Sarah	4
3.1	Ett utdrag fra resultatet av planningpoker for Sarah sine brukerhistorier	14
4.1	Tidlig utkast av database med oppgaver	18
4.2	Oversikt over sprinter i en tabell	19
4.3	Tabellvisning av sprint 1 sin backlog	19
4.4	Sprint 1 med tomt kanban board klar til å fylles	20
4.5	Alle referater føres i eget dokument	20
4.6	Burndown chart	23
5.1	Startside webbløsning	32
5.2	Enkelt produkt	34
5.3	Login side	35
5.4	Brukerside logget in	36
6.1	Sette root mappe	38
6.2	Laste pom.xml	39
6.3	Compound	39
6.4	Server og klient test	40
6.5	Trykk play	40
6.6	Terminalvinduer	40
6.7	Skrive inn klientID i terminal	41
6.8	Koble opp mot serveren	41
6.9	DeviceServer klassen	42

6.10	ServerSocket(9999)	42
6.11	ServerSocket(0)	43
6.12	Sette port med kommando - 1	43
6.13	Sette port med kommando - 2	43
7.1	Kompetansetest - Start	47
7.2	Kompetansetest - Velg et tall	47
7.3	Kompetansetest - Ett valg	47
7.4	Kompetansetest - Flervalg	48
7.5	Kompetansetest - resultat	48
7.6	Mine enheter - Dashboard	49
7.7	Mine enheter - Lys	49
7.8	Mine enheter - Støvsuger	49
7.9	Installasjonsguide - SKann produkt	50
7.10	Installasjonsguide - Koble til telefon	50
7.11	Installasjonsguide - Kobling vellykket	50
7.12	Installasjonsguide - Ferdig	50
7.13	Installasjonsguide - Avansert	51
7.14	Produktartikkel	52
7.15	Varslignssenter	52
7.16	Hjem Dashboard	53
7.17	Smart lås - aktivitet	54
7.18	Smart lås - tilganger	54
7.19	Klassediagram	54
7.20	EER-diagram	55
7.21	Heksagonal arkitektur[5]	56
7.22	Heksagonal arkitektur - UML	57

7.23	Login flytskjema	58
8.1	Produktliste	61
9.1	Kjøre tester i maven	71

Kode

6.1	Mock objekt for tester	44
6.2	Skritteller test 1	45
6.3	Funksjoner definert i smartWatch	45
6.4	Skritteller test 2	46
6.5	Skritteller test 3	46
6.6	Resultat av tester	46

Tabeller

3.1	Planning poker story points	13
3.2	Funksjonelle krav	15
3.3	Tekniske krav	16
3.4	Designmessige krav	17
4.1	Rollefordeling	21

Innledning

1.1 Om oppgaven

Denne rapporten er endelig levering i faget Software Engineering og Testing. I denne rapporten skal arbeidet gjort i gruppearbeidet vises. Oppgaven dreier seg om å kunne utvikle en løsning knyttet til smarthjem, og det har blitt gitt ganske mye frihet knyttet til eventuell løsning. Det skal fokuseres på prosjektstyring, metodikk, testing og utvikling av software.

1.2 Begrensninger

Vi har begrenset kunnskapsnivå når det kommer til nettverksprogrammering, avansert tråd-programmering, mobilprogrammering osv. Dette gjør at vi ikke kan fullføre alle typer oppgaver og at noen oppgaver tar lengre tid enn de burde siden vi må også lære oss basisen før vi setter i gang.

Dette er første fullstendige utviklingsprosjekt for mange på gruppen som gjør erfaringsnivået vårt ikke så høyt. Dette gjør det lettere å gå i feller og vanskeligere å estimere/prioritere rette oppgaver.

Vi har andre fag som har vært krevende for flere i gruppen som gjør at det kanskje er lagt inn mindre tid i prosjektet enn ønskelig.

1.3 Beskrivelse av klient

INkomplett er et nytt oppstartsselskap med ambisjoner om å bli den nye store nettbaserte elektronikk butikken i Norge. Et av bedriftens høyeste mål er å bli hovedleverandør for IoT produkter og tjenester.

Siden selskapet startet har de hatt en eksponensiell vekst i både kundebase og produsentavtaler. Per dags dato produserer ikke INkomplett noen egne produkter, men satser heller på å være et optimalt mellomledd mellom store produsenter og dens kunder. Dette har vært en god strategi, da de har fått flere gode avtaler med store elektronikk produsenter, som gjør at de kan selge produkter til en konkurransedyktig pris og fortsatt sitte igjen med en god bunnlinje for bedriften og dens ansatte.

Med tiden som kundebasen deres har vokst har de merket et stadig økende press på support avdelingen deres. Stadig flere frustrerte kunder tar kontakt fordi de enten ikke får produktet sitt til å fungere slik som ønsket, eller det produktet de har mottatt ikke var det de hadde forespeilet seg. De bemerker seg at majoriteten av sakene som kommer til support avdelingen er av relativt lav teknisk grad, men volumet gjør at bemanningen har store utfordringer med å besvare alle innkommende saker.

På ukentlige møter er det stadig forespørsler fra support om å få flere ansatte. Ledelsen ser at det absolutt foreligger et behov for å gjøre noe med dette, men bestemmer seg for å ta en vurdering på alternative løsninger før de går til steget med å ansette flere. Dette er hvor de bestemmer seg for å kontakte oss utviklere for å høre om vi har noen forslag på hvordan dette kan løses.

1.4 Beskrivelse av produktet

Vi blir møtt av en representant for bedriften INkomplett, som starter hele møtet med å forklare oss om den foreliggende problemstillingen med support avdelingen og hvordan de ønsker å avlaste avdelingen uten å måtte bite inn i budsjettet med en ny ansettelse.

Videre blir det forklart om hvordan majoriteten av henvendelser omhandler kunder som sliter med å installere produktet, frustrasjon over mangfoldet av applikasjoner fra forskjellige leverandører og retur av varer hvor kunde har kjøpt feil produkt til deres forbruk. Dette handler altså hovedsakelig om mindre tekniske utfordringer og manglende teknisk kompetanse hos klienten kunder.

Når representanten er ferdig med å forklare om situasjonen, åpnes det for en brainstorming sesjon for deltagerne i møtet. Ut fra dette ender vi opp med å foreslå en applikasjons løsning som er integrert med bedriften og dens IoT produkter. Hensikten med applikasjonen er at klientens kunder skal kunne få tilgang til et samlet sted som kan hjelpe med å installere og styre klientens IoT salgsprodukter, samt veilede klientens kunder via diverse medium på hvordan produktene brukes, og hvilke produkter som passer best til forbrukers behov.

Etter møtet er representanten for INkomplett svært positiv til forslaget og leier oss inn for å starte en mer detaljert fremstilling av applikasjonen, dens funksjoner og potensielle muligheter.

Brukerne

I dette kapitlet ønsker vi å se nærmere på tre forskjellige brukerne av det foreslåtte systemet. Vi vil gå inn på personas, lage scenario og en brukerhistorie for hver bruker.

2.1 Personas

For å finne de rette personene til å bruke systemet, så valgte vi å bruke generativ AI til å hjelpe oss med nettopp dette.^[1] Det ble spurt etter tre forskjellige brukere, hvor hver bruker har enten lav, middels eller høy teknisk innsikt til å bruke IoT enheter eller teknologi.

2.1.1 Sarah

Sarah Johnsen



ALDER: 62

YRKE: Barneskolelærer

SIVILSTATUS: Enke

TEKNISK FERDIGHETSNIVÅ: Lavt

Bakgrunn

Sarah har undervist i over 30 år. Hun vokste opp i en tid da teknologi ikke var en sentral del av livet. Selv om hun er komfortabel med enkel e-post og kan navigere i et tekstbehandlingsprogram, blir hun lett overveldet av alt som går utover dette.

Smertepunkter

Sliter med å forstå hvordan man bruker ny programvare, glemmer ofte passord, og føler seg usikker på å feilsøke selv enkle tekniske problemer. Hun stoler ofte på andre for å sette opp eller fikse ting på enhetene sine.

Teknologibruk

Bruker teknologi primært til enkle oppgaver som å sjekke e-post, ha videosamtaler med familien, og søke på nettet etter undervisningsplaner. Foretrekker fysiske bøker fremfor e-lesere og laster sjelden ned nye apper.

Figur 2.1: Personas - Sarah

2.1.2 Alex

Alex Martinsen



ALDER: 32

YRKE: Markedsførings-sjef

SIVILSTATUS: Samboer

TEKNISK FERDIGHETSNIVÅ: Middels

Bakgrunn

Alex er komfortabel med å bruke en rekke programvareverktøy i jobben sin, inkludert CRM-systemer, verktøy for sosiale medier og dataanalyseplattformer. Han er selvlært og generelt god til å finne løsninger på problemene sine på nettet.

Smertepunkter

Selv om Alex kan håndtere de fleste hverdagslige tekniske utfordringer, sliter han av og til med mer komplekse oppgaver som å sette opp et hjemmenettverk eller håndtere avansert feilsøking. Han er ikke alltid trygg på å identifisere de beste verktøyene for en gitt oppgave og kan bruke lengre tid enn nødvendig på å løse tekniske problemer.

Teknologibruk

Bruker teknologi daglig i jobb og privatliv, inkludert å administrere sosiale medier, analysere markedsdata og spille videospill. Han er komfortabel med smarttelefoner, datamaskiner og smarthus-enheter, men pleier å holde seg til det han kjenner og unngår dyper tilpasning.

Figur 2.2: Personas - Alex

2.1.3 Priya

Priya Dessai



ALDER: 28

YRKE: Programvare-ingeniør

SIVILSTATUS: Samboer

TEKNISK FERDIGHETSNIVÅ: Høyt

Bakgrunn

Priya har en grad i informatikk og har jobbet som programvareingeniør i seks år. Hun er dyktig i flere programmeringsspråk, forstår komplekse algoritmer, og kan bygge og feilsøke systemer fra bunnen av.

Smertepunkter

Priya blir ofte frustrert over begrensningene til forbrukerteknologi og foretrekker å bygge egne løsninger. Hun kan av og til overkomplisere ting for andre og må bevisst forenkle forklaringer når hun hjelper venner eller familie med tekniske problemer.

Teknologibruk

Bruker teknologi i stor grad, fra koding og utvikling av programvare til å drive egne servere og eksperimentere med kunstig intelligens. Priya har et fullt automatisert smarthjem, bidrar regelmessig til open-source-prosjekter, og er alltid på jakt etter de nyeste teknologitrendene.

Figur 2.3: Personas - Sarah

2.2 Scenario

For å bedre skjønne hver bruker ble det laget scenario for å beskrive brukerne litt bedre, og for å få en bedre innsikt i deres bruk, behov og forhold til IoT løsninger. Hver person fikk oppgave i å lage to scenario om hver person.

2.2.1 Sarah

2.2.1.1 Scenario 1

Selvstendig støvsuger – Sarah Johnsen har nettopp vært gjennom en hoftetransplantasjon og på grunn av dette har hun fått store problemer med daglige husholds aktiviteter hun har brukt å gjøre før. I denne sammenheng har hun blitt tipset om at det finnes gode produkter for å hjelpe henne med dette. Den første oppgaven hun har bestemt seg for å delegerer til en teknisk enhet er støvsugning, da leiligheten hennes har alt på en flate og på denne måten vil hun bare trenge en maskin for å potensielt slippe å støvsuge noe selv i tiden fremover. Fra før så har Sarah svært lite erfaring med IT relaterte produkter og finner alle apper og muligheter ganske overveldende. Hun er derfor svært usikker på hvilket produkt hun skal anskaffe, da hun ønsker å gjøre så lite som mulig av installasjon for at ting skal fungere som hun ønsker. Hun blir tipset om vår klient av en elev som hun er lærer for, hvor studenten betrygger henne med at om hun kjøper produktene siden gjennom denne leverandøren, så vil det følge med en enkel app som «holder» henne i hånden gjennom hele prosessen. Hun får også vite at alle potensielle fremtidige kjøp av vår klients produkter vil kunne håndteres i en og samme applikasjon.

Hun ender med å kjøpe en Samsung robotstøvsuger som vår klient har avertert for som et av sine topp produkter innenfor kategorien. I bestillings bekreftelsen får hun også en kort video introduksjon til mobil applikasjonen og en direkte link for å laste den ned på mobilen. Denne videoen viser henne i enkle steg hvordan hun kan installere applikasjonen på sin enhet og hva hun kommer til å bli møtt med når hun åpner den. En av funksjonene som hun la merke til var den promoterte AR-skann funksjonen til appen, hvor man bare kunne se på produktet eller dens forpakning for å komme til produktets introduksjonsside. Til tross for at dette virket litt for godt til å være sant, så bestemte hun seg for at hun skulle gi dette et forsøk når produktet ankom.

Dagen kommer når produktet ligger i esken sin i på stuegulvet til Sarah. Med skeptisisme åpner hun applikasjonen, trykker på AR-skann knappen og sikter kameraet mot esken. Etter noen sekunder endrer skjermen seg og hun befinner seg på en side i applikasjonen som omhandler produktet. Øverst står produktets navn som en klar tittel for å bekrefte at hun har kommet til rett artikkel, og under finnes en kort video og en kort punktliste om hvordan produktet skal settes opp før installasjonen kan starte, samt hvilke plasser det kan være lurt å plassere den. Når hun har gått gjennom disse stegene og ladestasjon og støvsuger står klart titter hun igjen på applikasjonen. Under oppsettet hun nettopp har gjennomført finner hun en stor knapp med teksten «start installasjon», over knappen står det «trykk her for å sette opp støvsugeren». Når hun trykker på knappen, starter mobilen automatisk en kobling med det gitt produktet og iverksetter de nødvendige konfigurasjoner. Etter at dette er fullført får Sarah opp en liste over ønskede preferanser, slik som tider støvsugeren skal operere på. Støvsugeren durer i gang og Sarah behøver ikke tenke mer på denne husholdsoppgaven. Støvsugeren

legger seg automatisk i en liste over Sarah sine produkter, som blir en snarvei på applikasjonens forside.

2.2.1.2 Scenario 2

Problem

Det er frost ute, og prognosene viser nok en uholdbar vinter når Sarah skrapper is av sin gamle fossilbil en kald november morgen. Som i eget hjem lider også barneskolen hun underviser av å være et gammelt bygg med dårlig isolasjon og lav temperatur. For å unngå økonomiske underskudd må skolen redusere strømforbruket. Sarah deler ut lapper til foreldrene ved skoledagens slutt som informerer om tiltakene, men forsikrer samtidig om at skolen tar situasjonen på alvor ved å investere i et smart system som sikrer en stabil temperatur for elevene.

Motivasjon

Sarah er også interessert i å redusere kostnadene i eget hjem, men faller fort av når de tekniske detaljene diskuteres på lærerværelset. En forelder som er elektriker, tipser henne om et system som kan anbefale og veilede henne gjennom installasjon av smarte IoT-produkter basert på hennes egne preferanser. Sarah blir vist en enkel app som gir en «quick start»-guide for å fylle inn grunnleggende informasjon om hjemmet sitt og hennes mål for strømsparing.

Handling og gjennomføring

Appen anbefaler kjøp av termostatstyrte enheter som kan kobles til strømuttakene for blant annet varmeovner og varmtvannsberedere. Disse ligner på de enhetene som ble omtalt på arbeidsplassen, og hun blir interessert i å lese mer siden appen fremhever kun nøkkelinfo som hun vil vite. Produktartikkelen inneholder også en knapp som tar henne direkte til salgsiden for dette produktet, som drives av et oppstartsselskap som selger ulike IoT-smarthusløsninger. Når pakken ankommer hjemmet, trenger Sarah bare å skanne strekkoden, og en full installasjonsguide blir tilgjengelig, inkludert en sjekkliste og forklarende enkle videoer. Hun kan også teste at enhetene fungerer med Wi-Fi-nettverket og opp mot produktets egne systemer for at alt snakker riktig sammen. Ved å følge de enkle instruksjonene har hun nå spart mye penger både på strømforbruk, unngå profesjonell hjelp til installasjon, og på kjøp av bedre varmeovner.

2.2.2 Alex

2.2.2.1 Scenario 1

Problem

Alex og samboeren lever et hektisk liv med mye reising på grunn av jobbene sine. Dette skaper utfordringer med å passe på deres to katter, som ofte må ha tilsyn av familie og venner når de er borte i 2-4 dager av gangen. De bor i et tettbygd boligområde, og kattene er avhengige av å kunne gå ut og inn når de vil. Samboeren er svært opptatt av kattenes helse og synes det er ubeleilig å

stadig måtte be venner om hjelp. Begge er middels teknisk anlagt og håper at moderne teknologi kan løse problemet med å ta vare på kattene.

Alex er flink til å markedsføre produkter på jobben, men blir frustrert når det kommer til å sette seg inn i tekniske detaljer hjemme og når alt skal ha sin egen app på telefonen. Paret bruker sosiale medier aktivt både privat og på jobb, og blir ofte eksponert for annonser om smarte IoT-løsninger for husdyr, som selvfylgende matskåler, automatiske katteluker, GPS-sporing, og katteleker.

Motivasjon

En dag ser Alex en annonsevideo fra et ny oppstartet selskap som viser hvordan IoT-produkter kan gjøre hjemmet smartere. Et av klippene i videoen handler om løsninger for husdyr, noe som vekker Alex sin interesse. Selv om han er skeptisk til det nye selskapet på grunn av manglende kredibilitet, klikker han seg videre fordi nettsiden tilbyr kun en enkelt app for både veiledning, installasjon og styring av alle enheter i samme pakke. Handling og gjennomføring

På nettsiden kan paret gå gjennom en enkel veileder hvor de beskriver sitt kompetansenivå med IoT og hva de ønsker å oppnå i hjemmet. Etter å ha fylt inn informasjon om kattene og deres livsstil, får de anbefalinger om IoT-pakker som er tilpasset husdyr, med trinn-for-trinn installasjonsvideoer, anbefalt forkunnskap og fordeler for dyrene. De bestemmer seg for å kjøpe en startpakke, men blir raskt overveldet når hver enhet krever egen Wi-Fi-tilkobling og oppsett. Heldigvis tilbys det en løsning der de kan ta bilde av hver enhet og få konfigurere direkte på telefonen, uten å måtte lete opp hver produsent. Ved å følge denne guiden klarer de å sette opp systemene uten større problemer.

Konklusjon

Gjennom denne prosessen har Alex og samboeren løst utfordringen med å ta vare på kattene når de er borte, uten å måtte involvere venner og familie. De har fått et system som automatiserer tilsynet av kattene, sparer tid og reduserer stress i en travel hverdag. Dette har også gitt Alex en følelse av mestring når det gjelder teknologi, og ført til en smartere og mer bekymringsfri hverdag for dem begge

2.2.2.2 Scenario 2

Alex er en dyktig og selvlært middels IT-kyndig mann på 32 år. Han jobber som markedsføringssjef innenfor et av de mest voksende virksomheten i området, når det kommer til teknologi og IT. Alex er veldig komfortabel med all slags typer elektronikk både hjemme og i jobbsammenheng.

Et par eksempler som er verdt å nevne om han er blant annet at han bruker sosiale medier på jobben der han administrerer bedriftsprofilen til arbeidsplassen, samtidig som han analyserer data.

Hjemme surfer han også på mange forskjellige plattformer når han får lyst, spiller spill og hjelper folk en gang i mellom på forumer der han kan stille med litt IT-hjelp. Han er altså veldig komfortabel med smarttelefoner, pc'er og smarthus-enheter men er ikke så villig til å gå mer i dybden på elektronikken enn det han må/kan.

Alex er ikke akkurat en profesjonell i feltet sitt. Han er selvlært, dette betyr at han kan nok om det han bør kunne og det som har interessert han under sin egen opplæring og ikke mye mer. Veldig bred kunnskap var ikke på agendaen. Eksempelvis sliter han av og til med større komplekse oppgaver som å håndtere avansert feilsøking og sette opp egne nettverk på nett, og det å finne riktig verktøy for å løse et større teknisk problem, bruker han en del tid på.

En dag når Alex surfet på forskjellige forumer for å hjelpe folk, begynte han å merke et klart mønster. De fleste spørsmålene folk stilte rundt smarthus elektronikken var det ganske enkle løsninger på. Det meste var tilkoblinger og app komplikasjoner som forvirret de fleste. Dette fikk han til å tenke på den IoT tjenesten han hadde sett på nettet hvor noen hadde delt en reklame som hadde gjort livet mye enklere for mange av disse brukerne. En enkel applikasjon som samlet alle IOT enheter på et sted. Oversiktlig, enkel, interaktiv med store bilder, mange farger og et enkelt brukergrensesnitt som gjør det enkelt både for yngre og eldre samt de med litt vansker, å faktisk ta i bruk denne applikasjonen.

Tjenesten tilbyr en løsning for alle typer mennesker. Svaksynte, teknisk uerfarne eller litt bedre teknisk erfarne. Det fins et «nivå» for alle. Alex noterer ned denne tjenesten med en gang slik at han husker å nevne det til sine nærmeste venner og familie, om de skulle kjøpe produkter som kunne ha behov for dette.

2.2.2.3 Scenario 3

Som markedsføringssjef har Alex Martinsen bygget seg opp en generell forståelse på IT systemer, jobben hans krever også at han henger med på markedet. Martinsen har allerede smart lyspærer og bruker det ofte for å vise fram hvor teknisk han er til besøk.

Han er blitt nysgjerrig på IOT enheter i det siste og prøve seg med mange forskjellige enheter. Han tenker at det ikke skal bli noe problem han er jo tross alt god på å fikle med teknologi. Han bestemmer seg for å anskaffe en smart kaffemaskin, en smart gressklipper, en smart garasjeport, ett smartlås system og en robotstøvsuger. Alex bruker flere dager på å søke informasjon om enhetene, men han er veldig usikker på hva som blir det rette for han. Han skulle ønske det ikke fantes så mange forskjellige leverandører, for da hadde jo valget vært lett. Etter å ha lett i en uke har han endelig fått bestilt alle enhetene. De er alle fra forskjellige leverandører, men Alex tenker ikke over at dette kan by på problemer. Når han får sin første enhet, får han raskt satt den opp og alt ser til å fungere fint. Men etter hvert som han får flere enheter må han laste ned stadig flere applikasjoner.

Mobilen er nå full av forskjellige applikasjoner som alle fungerer på forskjellige måter. I tillegg har han greid å sette støvsugeren på tidsskjema til gressklipperen, og døra til å være ulåst basert på støvsugeres vasketider. Han føler hele huset lever sitt eget liv nå, og når låsen begynner å pipe blir det for mye.

Han legger flere sinte innlegg ut på nettet angående hvor elendige disse smart dingsene er. I kommentarfeltet er det mange som er enige med han. Det er en kommentar som skiller seg ut. Den henviser til en applikasjon som skal kunne gjøre det lettere å bruke IOT enheter. Alex tenker at han har ikke så mye å tape på å laste den ned og spesielt siden det står at applikasjonen skal kunne samle enhetene på en plass.

Etter å ha gått gjennom applikasjonen viser det seg at han har kjøpt de minst compatible enhetene og at ved å bytte enkelte av enhetene kan han få samlet alle enhetene i en meny. I tillegg er oppsettet veldig likt. Applikasjonen lar han også se på andre enheter som kan være av interesse og hjelper han med å ta det vanskelige valget med å velge rett enhet.

Alex blir sakte men sikkert en ekspert på IOT dingser i sitt hjem, og forteller storartet alle som besøker om hvor teknisk kompetent han er, siden han har satt opp alle disse dingsene helt selv.

2.2.3 Priya

2.2.3.1 Scenario 1

Teknisk avlastning – Priya er en aktiv programmerer med en hverdag som holder et høyt tempo. På en daglig basis sitter hun med ett eller flere prosjekt på sine maskiner. Hun jobber som frilanser og har ofte mange deadlines som hun må opprettholde. Hjemmet hennes er tilnærmet helautomatisert etter eget design og hun kontrollerer enhetene sine på en egenutviklet plattform som er skreddersydd til hennes kompetansenivå.

Dette fungerer svært fint for henne da hun er godt kjent med sin egen programvare. Derimot er ikke denne løsningen like gunstig for partneren hennes som nylig har flyttet hjem til henne. Denne personen er særdeles mindre teknisk enn henne, og allerede første uken har de kommet i konflikt da partneren ikke får til å lage seg så mye som en kopp kaffe uten å spørre henne om å starte maskinen. Dette er svært frustrerende for paret, da hun ikke har tid til å bli avbrutt i arbeidet sitt hele tiden og partneren ikke er interessert i å lære seg den intrikate plattformen hun selv har utviklet.

Priya som har svært god oversikt over de tilgjengelige løsningene som finnes på markedet ender opp med å ta en nærmere titt på applikasjonen som vår klient tilbyr, da de averterer med at denne løsningen er satt opp med et allment tilnærmingnivå for daglig bruk av IoT enheter. Etter å ha inspisert programvarens arkitektur, ser hun at hun enkelt kan eksportere sine personlige konfigurasjoner på de nødvendige IoT enhetene til denne applikasjonen.

Hun starter med å gjøre dette på sin egen enhet for å teste. I prosessen finner hun ut at den personlige enhets menyen i applikasjonen er svært intuitiv og inneholder også spesifikt produktinformasjon som er vesentlig for henne når hun kjører applikasjonen med en avansert profil.

Dette leder til at hun selv begynner å bruke applikasjonen for å rask slå opp informasjon om sine enheter når hun ikke sitter foran hoved maskinen sin som er en desktop PC. Etter dette gjør hun samme prosess på partneren sin mobil enhet, men setter profil nivå til mindre avansert. Dette fører til at partneren får en tilsvarende liste over enheter i sin applikasjon, men uten den tekniske informasjonen og heller en enklere styring av enhetene, slik som start/stopp og tidsskjema konfigurering.

Etter denne oppdagelsen kan nå partneren endelig bruke IoT enhetene i leiligheten uten å måtte spørre om hjelp, og Priya kan gjøre enkle status sjekker, oppdatering samt finne relevant teknisk informasjon om enheten uten å måtte sette seg på kontoret.

2.2.3.2 Scenario 2

Priya er en 28 år gammel programvareingeniør med høy teknisk kompetanse. Dette gjør at hun både er teknisk aktiv hjemme hos seg selv og på jobb.

Hjemme hos seg selv har Priya satt opp alt slik at det meste styres med smarthus-teknologi og IoT systemer for å gjøre hverdagen sin enklere og mer spennende. På jobb er hun dyktig i å bygge og feilsøke systemer fra bunnen av.

Svakheten hennes er det at hun med sin brede tekniske kompetanse, har en vane til å overkomplisere ting for andre og glemmer at de må ha det brutt ned for å forstå det. Priya blir fort frustrert over forbrukerteknologi og mener det er enklere å bare fikse ting selv med egne løsninger. Priya sliter spesielt med å hjelpe besteforeldrene sine med tekniske smarthus produkter som støvsugere, gressklippere tv'er osv.. Hun vet ikke hvor mye enklere forklaringer hun er i stand til å gi.

En dag mens Priya var på jobb, kom en kollega bort mens de hadde kaffepause. Alex, sjefen for markedsføringsavdelingen. Her begynte frustrerte Priya å snakke om at hennes besteforeldre sliter veldig med IoT systemer og at hun ikke får hjulpet dem på en effektiv måte. Da anbefalte han en applikasjon han hadde sett reklame for, og selv bruker for å hjelpe bekjente. Løsningen han snakket om var utviklet til akkurat disse type smarthus-enhetene. Den er enkel, intuitiv og veldig brukervennlig når det kommer til brukergrensesnittet og enhets instruksjonene.

Applikasjonen gir deg muligheten til å velge hva slags type teknisk person du er og hva du eventuelt har vansker med, for å tilpasse seg brukeren. Stor tekst, bilder og farger med en god interaksjon gjør det spennende å få satt opp de ulike elektroniske enhetene. Dette syns Priya hørtes spennende og smart ut så hun ba om linken til produktet for å ta en videre titt når hun hadde mulighet.

Etter å ha investigert produktet Alex anbefalte henne, tenkte hun å ta det opp med besteforeldrene sine neste gang hun var der. Da dagen endelig kom for å hjelpe de med å linke den elektriske gressklipperen opp mot applikasjonen på telefonen, tok Priya opp produktet hun hadde sett på.

Hun forklarte dem om løsningen, samt de ulike nivåene som er tilpasset personer med forskjellig alder og vansker, hvorfor de burde ta en titt og ikke være redde for det. Etter litt fram og tilbake ble besteforeldrene fornøyde. De likte spesielt at Priya, som er så teknisk, satt med dem og forklarte hvordan de enkelt kunne få til å navigere rundt i applikasjonen, og vise hva de ulike tingene gjorde. De ble solgt på stedet.

Dette gjorde hverdagen til besteforeldrene bedre og ga Priya mer tid til andre ting fremfor å dra til besteforeldrene for de minste tekniske problemer.

2.3 Brukerhistorier

2.3.1 Sarah

Som Sarah Johnsen vil jeg enkelt kunne sette opp og bruke smarte enheter i hjemmet mitt, slik at jeg kan redusere energiforbruket, håndtere daglige husholdningsoppgaver uten fysisk belastning, og få mer tid og overskudd til å slappe av etter en lang arbeidsdag.

Akseptansekriterier:

- Sarah kan bruke en enkel app for å legge inn informasjon om hjemmet sitt og sine mål for energibesparelse og husholdningshjelp.
- Appen gir henne anbefalinger om spesifikke smarte IoT-enheter, som termostatstyrte stikkontakter til varmeovner og en robotstøvsuger tilpasset hennes behov og leilighetens størrelse.
- Produktene kan kjøpes direkte gjennom appen, og Sarah mottar en video og AR-basert veiledning som viser hvordan enhetene settes opp på en enkel måte.
- Appen guider Sarah gjennom installasjonen, konfigurerer automatisk enhetene, og gir klare instruksjoner om hvordan hun kan sette opp og justere innstillingene, som tidsskjema for støvsugeren. Sarah får varsler fra appen hvis enhetene trenger oppmerksomhet. Slik som om støvsugeren setter seg fast eller hvis energiforbruket kan optimaliseres.
- Etter installasjonen kan Sarah enkelt kontrollere alle smarte enheter fra appen. Noe som gir henne trygghet og mer fritid til å slappe av uten å bekymre seg for husarbeid eller tekniske utfordringer.

2.3.2 Alex

Som Alex Martinsen ønsker jeg å sette opp et felles system for alle smartenhetene jeg har, hvor jeg enkelt kan administrere enheter fra en portal.

Kriterier:

- Skal kunne koble seg til eksisterende smartenheter i hjemmet. Disse bør kunne administreres og overvåkes fra en portal.
- Ved feil bør systemet kunne gi beskjed. Forslag til forbedringer eller utstyr som må skiftes bør også informeres om.
- Portalen bør være enkel i bruk, men kunne ha mulighet til å finjustere og stille inn avanserte funksjoner. Bør følge moderne designprinsipper og være estetisk fin. Helst ha en løsning til mobile enheter, men også en webapplikasjon
- Systemet bør være sikkert og enhver digital løsning bør ha 2faktor autentisering og motta kontinuerlige oppdateringer slik at den er trygg mot digitalt angrep. Er ikke interessert i å miste kontrollen over smartenhetene
- Produsenten burde ha gode hjelpesider og støtte på nett sånn at man lett kan lese seg opp på det man trenger. Gjerne egne forum som man kan lese på. Hvis det ikke er mulig å finne løsning selv så bør kundeservicen kunne hjelpe en når en trenger hjelp

2.3.3 Priya

Som Priya Desai vil jeg kunne sette opp et smart-system for mine besteforeldres hjem slik at de ikke trenger å bekymre seg for tekniske problemer fremover, og jeg får mer tid til andre ting i hverdagen.

Kriterier:

- Applikasjonen skal ha et enkelt intuitivt brukergrensesnitt, med justerbare størrelse på tekst og knapper, tydelige ikoner og lettlest tekst som skal gjøre det enkelt for besteforeldrene å navigere.
- Den skal inneholde trinnvise veiledninger der man kan velge å enten lese, eller følge en instruksjons video.
- Enkelt oppsett, og tilkoblingsprosess.
- Vanskelighetsnivåer for ulike typer brukere. Applikasjonen vurderer brukeren med noen korte spørsmål de skal svare på for å gi dem opplevelsen som passer deres stil best.
- Applikasjonen skal fremvises slik at det er enkelt å få tilgang til kundeservice og support ved behov. Telefon og livechat er begge gode muligheter.
- Priya har fått banket inn i øret at personvern er høyt verdsatt av besteforeldrene. De lurer stadig på hvor deres personlige data ender opp etter registreringer. Så sikre personvern er også en god funksjon å ha på plass

Kravspesifikasjon

3.1 Planning poker

Planning poker ble gjennomført for å skape en diskusjon rundt brukerhistorien, samt ett veldig røft estimat av hvor lang tid oppgavene knyttet til brukerhistoriene vil ta. Vi brukte [Scrum poker-online](#) som verktøy for å utføre planning poker".

Pokeren ble gjennomført ved at en enkel brukerhistorie ble lest opp, så en kort diskusjonsrunde hvor vi diskuterte eventuelle uklarheter. Vi valgte så kort med en gitt mengde storypoints, for så å avsløre alle kortene samtidig. Deretter ble det en diskusjon rundt hva alle så for seg lå i brukerhistorien og hva slags features den inneholdt. Så avsluttet vi med å finne en mengde storypoints som alle kunne stille seg bak.

Tabellen under viser hvordan vi vektla de forskjellige storypoints. Da det skulle velges kort skulle alle 3 kategoriene tas hensyn til.

Estimert innsats	x > 2 timer	2-4 timer	5-8 timer	1-2 dager	2-4 dager	4-7 dager	x < 7 dager
Vår kunnskap	alt	nesten alt	mye	noe	lite	veldig lite	ukjent
Avhengigheter	ingen	svært få	lite	en del	mye	veldig mye	alt
Storypoints	0.5	1	2	3	5	8	13

Tabell 3.1: Planning poker story points

Gjennomgangen startet ganske langsomt da vi diskuterte for detaljert under den første diskusjonsfasen. Etterhvert som vi gikk gjennom flere runder ble vi mer effektive. Siden brukerhistoriene er ganske brede, og kan inneholde flere features var det vanskelig å estimere innsats. Dette antar vi kommer til å bli lettere senere når vi skal bryte kravene ned i mindre oppgaver. Diskusjonene hjalp oss også med å identifisere features fra brukerhistoriene.

Brukerhistorier for Sarah

	Arshad	Dana/Andreas	Marius	Kjell-Magne	Felles sum	Resultat
Brukerinput om hjem og energisparing	8	13	3	8	8	Splitte opp i to oppgaver, en for info om hjem, en for energisparing
Anbefaling av IoT enheter basert på input fra bruker	5	5	5	5	5	
Nettbutikk, video og AR-basert veiledning	8	13	13	13	13	Splitte nettbutikk og AR scan i 2 forskjellige features
Installasjonsguide, konfigurasjon og plan for produkt	13	8	8	13	8	Splitte opp installasjonsguide, konfigurering og schedule plan for produktet i hver sin feature
Varsling fra IoT enhet	8	5	3	5	5	Gitt at vi har tilgang på API til IoT enhet tar det relativt kort tid å implementere.
Kontrollpanel for brukers IoT enheter	5	13	5	8	8	Gitt at hver kategori av IoT enheter kan ha ulike funksjoner lander vi på 8er

Figur 3.1: Ett utdrag fra resultatet av planningpoker for Sarah sine brukerhistorier

3.2 Features

Etter at gruppen sammen hadde blitt enige om noen gode scenarioer, gikk vi løs på brukerhistorier der vi fant ut av hva de spesifikke behovene for kundegruppen vår potensielt kunne være. Det ble da litt klarere for oss om hva slags type funksjonalitet vi skal ta for oss som potensielle og realistiske ‘features’. Dette er nødvendig slik at vi kan filtrere ned og mer nøyaktig vite hva vi skal flette med oss videre inn som kravspesifikasjon. Deretter ble de satt inn i 3 forskjellige kategorier for kravspesifikasjoner, nemlig ‘Funksjonelle krav’, ‘Tekniske krav’ og ‘Design krav’.

Det ble mange forskjellig funksjoner ut av brukerhistoriene. Vi fikk opp imot 25 punkter + et ekstra ‘innloggingspunkt’. Enkelte av funksjonalitetene våres gikk litt inn i hverandre, og noen hadde litt større teknisk funksjonalitet enn andre. Når dette var tilfellet valgte vi å dele opp funksjonaliteten i mindre deler og ta det videre derfra. Funksjonalitetene våres viste seg til å være alt fra kjøp og nettbutikk i app, database, push varslinger, litt om applikasjonens sikkerhet, FAQ’s og personvern, med mye mer.

Det er åpenbart forskjellig nivå på kompleksitet til enkelte av funksjonene. Derfor har vi gjennom Planning Poker – metoden senere tatt i betraktning hvor lang tid noen av funksjonene og implementasjonene kan ta. Det er sannsynlig at enkelte idéer ender opp som honorable mentions eller at det blir ideer for videreutvikling etter at produktet er produsert ferdig for forbruker.

3.3 Krav

3.3.1 Funksjonelle krav

#	Krav	Beskrivelse
1	Bruker profildanning(input)	Bruker fører inn relevant info om hjemmet som brukes til produktanbefalninger(TK #1)
2	Målsetting av strømforbruk	Kunde får en oversikt over sitt strømforbruk og kan sette mål for fremtidig forbruk
3	Dependencies list	Kunde får en liste(TK #3) over hvilke avhengigheter som er til produktet.
4	Installasjons guide	En guide som starter opp når kunde skal installere et produkt.
5	Link til nettbutikk	I artikkel for produktet vil det foreligge en link som tar kunde til produktet i nettbutikken.
6	Scann produkt	En knapp som lar kunde scanne produktet via en AR/bar/qr-løsning(TK #4)
7	Enhetstyring	Kunde får tilgang til et panel hvor de kan styre den gitte enhet.
8	Varsel/ping funksjoner	Kunde får push notifikasjoner om produktets status.
9	Justering av varsling.	En mulighet for kunde til å konfigurere applikasjonsvarslinger.
10	produkt artikkel	En artikkel som omfatter produktet og installasjon (mulighet for mer).
11	Avansert/enkelt oppsett	Toggle knapp som endrer mellom avansert og enkelt oppsett i applikasjon.
12	Bruker sikkerhet	Avgrense brukers muligheter som gjør at bruker ikke setter sine personlige data i fare.
13	Kompetansetest	Et spørreskjema ved førstegangsbruk av applikasjon for å determinere brukers kompetansenivå.
14	Tillatelser (GDPR)	Spør bruker om tilgang til relevante funksjoner for at applikasjonen skal kunne utføre gitte handlinger.
15	Login/Create user	Startside hvor kunde får mulighet til å logge inn/opprette bruker.

Tabell 3.2: Funksjonelle krav

3.3.2 Tekniske krav

#	Krav	Beskrivelse.
1	Produktanbefalninger	Program som gir ut en liste over anbefalte produkter til kunden basert på bruker profildanning(FK #1)
2	Monitorering av strømforbruk	Programvare som som henter data fra et leserprodukt i kundens sikringsskap.
3	Dependencies generering	Genererer en liste over produktets avhengigheter.
4	Gjenkjenn enhet	Gjenkjenne enhet basert på enten AR eller Bar/QR-kode skanning
5	Database	En database for å lagre informasjon
6	Installasjon protokoll	Bakenforliggende prosesser som kjøres gjennom installasjons guide(FK #4)
7	Enhets API	Gjør bakenforliggende handlinger som gjør Enhetsstyring(FK #7) funksjonibelt
8	Status monitorering	Bakenforliggende program som henter ut status koder fra kundens aktive enheter.
9	System sikkerhet	Utvikle sikkerhet med hensyn til kundes sikkerhet(hacking osv.). Oppfylle ISO-sertifisering til beste evne.
10	Analyse av kompetansenivå	Et bakenforliggende program som evaluerer brukers kompetansenivå og setter profilnivå deretter.(FK #13)
11	Funksjonstilganger	Bakenforliggende program som gir tilgang til de funksjoner bruker tillater, for applikasjons funksjonalitet.
12	Cookies/sessions(datalagring)	Lagre data lokalt på enheten for å holde bruker innlogget(FK #15)

Tabell 3.3: Tekniske krav

3.3.3 Designmessige krav

#	Krav	Beskrivelse.
1	Skrivebord/home	En startside for applikasjonen.
2	“Mine produkter” meny	En liste over kundens installerte produkter, med tilhørende kontrollpanel for enkelte enheter(FK #7/PK #7).
3	GUI design	Etter moderne UI og UX prinsipper.

4	Universell utforming	Gjøre applikasjonen tilgjengelig for brukere med nedsatt funksjonsevne.
---	----------------------	---

Tabell 3.4: Designmessige krav

Utviklingsprosessen

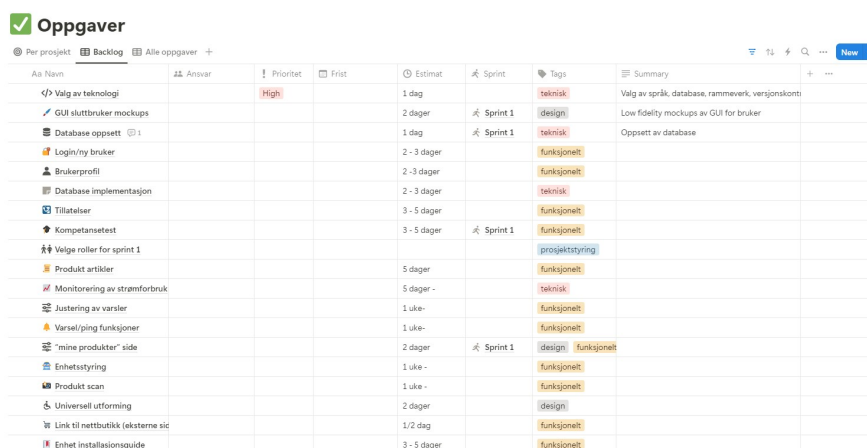
I dette kapittelet vil det blir diskutert og vist hvordan utviklingsprosessen har vært. Her vil det også reflekteres over prosessen, og samtidig vise til en vurdering av siste sprint.

4.1 Prosjektstyring

4.1.1 Prosjektstyringsverktøy - Notion

På grunn av fleksible muligheter for samarbeid ble Notion valgt som prosjektstyring verktøy. Med mange muligheter tilfører også en bratt læringskurve, men av medlemmene hadde tidligere erfaring med Notion som gjorde det enklere å sette de andre inn i verktøyet. Notion ble brukt til å sette opp en database med alle oppgaver som skulle løses, som automatisk la seg i en «backlog», som vist på figur 4.1.

Link til Notion kan finnes [her](#)



Aa Navn	Ansvar	Prioritet	Frist	Estimat	Sprint	Tags	Summary
</> Valg av teknologi		High		1 dag		teknisk	Valg av språk, database, rammeverk, versjonskontroll
GUI sluttbruker mockups				2 dager	Sprint 1	design	Low fidelity mockups av GUI for bruker
Database oppsett				1 dag	Sprint 1	teknisk	Oppsett av database
Login/ny bruker				2-3 dager		funksjonelt	
Brakerprofil				2-3 dager		funksjonelt	
Database implementasjon				2-3 dager		teknisk	
Tilfater				3-5 dager		funksjonelt	
Kompetanasett				3-5 dager	Sprint 1	funksjonelt	
Velge roller for sprint 1						prosjektstyring	
Produkt artikler				5 dager		funksjonelt	
Monitering av stramforbruk				5 dager -		teknisk	
Justering av varsler				1 uke -		funksjonelt	
Varsel/ping funksjoner				1 uke -		funksjonelt	
"mine produkter" side				2 dager	Sprint 1	design funksjonelt	
Enhetsstyring				1 uke -		funksjonelt	
Produkt scan				1 uke -		funksjonelt	
Universell utforming				2 dager		design	
Link til nettbutikk (eksterne side)				1/2 dag		funksjonelt	
Enhet installasjonsguide				3-5 dager		funksjonelt	

Figur 4.1: Tidlig utkast av database med oppgaver

Det ble så opprettet linkede utgaver av databasen som kan filtrere oppgaver på sprinter, eller en viss arbeidsperiode. Figur 4.2 viser en tabell med alle sprinter, hvor sprint 0 og 1 følger samme tidsperiode som innlevering 1 og 2. Ved å gå inn på en sprint kommer det opp en tabell med oppgaver som er tildelt den aktuelle sprinten, som vist i figur 3. Tabellen viser i tillegg felter som hvilken sprint er aktiv, planlagt dato, antall oppgaver sprinten er tildelt og progresjon over de som er markert «done»

Alle sprints
Learn how to use Sprints

📅 All 📅 Timeline +

Aa Sprint name	🔍 Sprint status	📅 Dates	📊 Total tasks	📈 Completed tasks	+ ...
🚀 Sprint 0	● Current	September 1, 2024 → September 8, 2024	13	69.2%	
🚀 Sprint 1	● Next	September 9, 2024 → September 22, 2024	4	0%	
🚀 Sprint 2	● Future		0		
🚀 Sprint 3	● Future		0		
🚀 Sprint 4	● Future		0		

+ New

Calculate ▾

Figur 4.2: Oversikt over sprinter i en tabell

Ved å gå inn på en sprint kommer det opp en ny tabell med listevising av oppgaver som sprinten tilhører. Figur 4.3 viser at hver sprint også har sin egen backlog, oppgaver som skal gjøres i den perioden, men ikke er tildelt ansvar eller påbegynt. Oppgavene egenskaper som gjør de enklere å strukturere, som hvem er ansvarlig, estimat, legge til tagger og status.

Sprint 1

🔍 Sprint status ● Next

📅 Dates September 9, 2024 → September 22, 2024

📈 Completed tasks 0%

▽ 2 more properties

🗨 Add a comment...

🚀 4 Tasks

📅 Board view 📅 Backlog

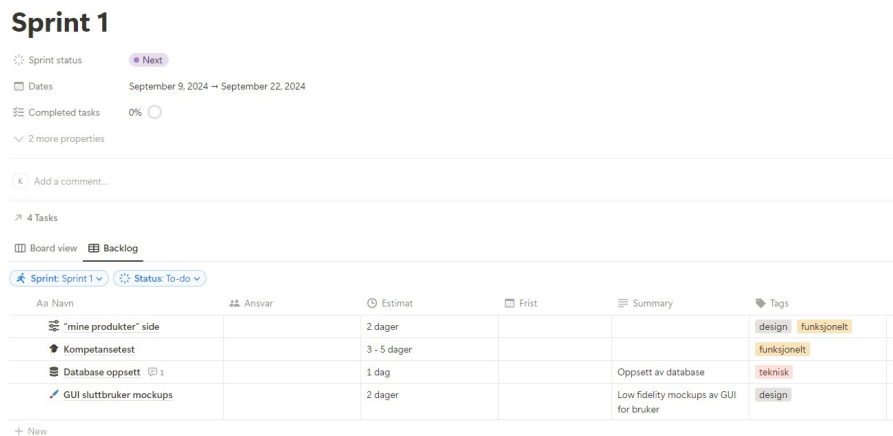
🚀 Sprint: Sprint 1 ▾ 🔍 Status: To-do ▾

Aa Navn	👤 Ansvar	🕒 Estimert	📅 Frist	📋 Summary	🏷 Tags
📁 "mine produkter" side		2 dager			design funksjonelt
🚀 Kompetanasett		3 - 5 dager			funksjonelt
📁 Database oppsett		1 dag		Oppsett av database	teknisk
🚀 GUI sluttbruker mockups		2 dager		Low fidelity mockups av GUI for bruker	design

+ New

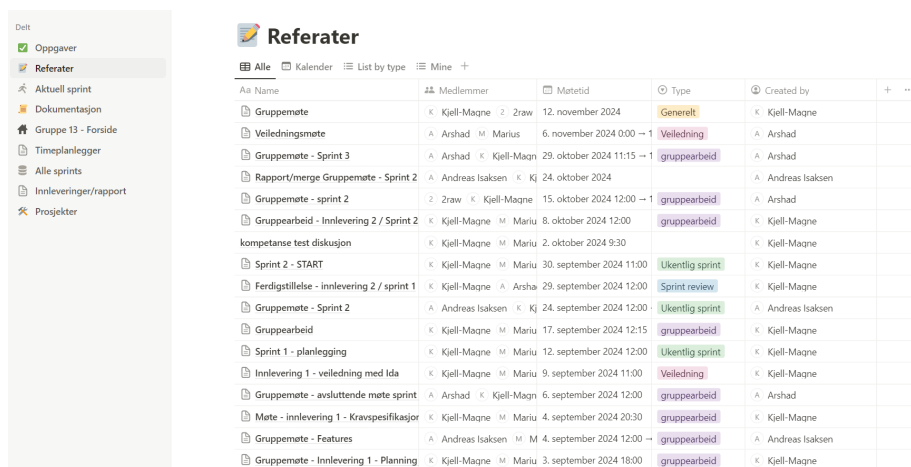
Figur 4.3: Tabellvisning av sprint 1 sin backlog

Ved å endre oppgaven sin status til «in progress» blir den flyttet ut av backlog og inn i et kanban board. Figur 4.4 viser en egen tabell med kort som kan flyttes rundt i egne kolonner hvor hver kolonne markerer om oppgaven er under arbeid, trenger tilbakemelding, eller ferdig. Det kunne sikkert vært lagt til flere statuskoder, men det holdt for å minimere arbeid som brukes til å administrere Notion. Før en oppgave ble markert som «done» måtte flere på gruppa godkjenne at arbeidet var gjort tilstrekkelig. Når det er så mange medlemmer på gruppa var det viktig for alle å vite hva som var gjort og kvaliteten av det som leveres.



Figur 4.4: Sprint 1 med tomt kanban board klar til å fylles

Hver oppgave kunne åpnes som et eget dokument, der kunne en skrive notater, to-do liste, legge ved bilder, kode, eller andre relevant innhold. Notion ble også brukt til å føre referater fra møter. Figur 4.5 viser en liste over referater som er ført av weeklies, standups, veiledning og generelt gruppearbeid. Det har vært viktig å føre referater på en slik måte at gruppemedlemmer som uteblir fra møte kan få en oversikt over hva som ble gått gjennom.



Figur 4.5: Alle referater føres i eget dokument

4.1.2 SCRUM

Prosjektgruppen har etter beste evne forsøkt å jobbe etter SCRUM metoden for å etterkomme kundens ønsker og behov. Gruppen ønsket å være innom alle punktene så mye som mulig for å tilpasse seg en smidig tilnærming å jobbe mot. Allerede i sprint 0 var prosjektstyringsverktøy satt opp med artefakter, inkludert produkt backlog, sprinter og sprint backlog og mulighet for produkt inkremering. Det ble lagt til rette for å jobbe i sprinter som for ordens skyld ble tilpasset innleveringsperiodene.

Hver sprint startet med å holde retrospekt møte for den forrige sprinten. Det ble diskutert hva medlemmene har produsert i perioden, hva gikk bra og hva har bedre potensial. Møtet var nyttig for å kunne forutse hvilke krav som ville bli for store og mulig stykkes opp i mindre oppgaver for å opprettholde korte estimer. Planleggingsmøte for neste sprint ble holdt like retrospekt. Der ble det vedtatt hvilke oppgaver, nye eller eksisterende som skulle få bli med videre. Roller ble delt og hvert medlem kunne selv velge hvilke krav en ville jobbe med. Som regel ble oppgavene valgt basert på kunnskapsnivå og interesse, men gruppa prøvde å legge til rette for at alle skulle prøve seg inne hvert felt.

For å balansere andre fag og jobber ble daglige standups sløyfet, men gruppa ønsket å jobbe så mye som mulig å plenum når det var relevant, fysisk på skolen eller digitalt over Discord. I avsnittene 4.4.1, 4.4.2 og 4.4.3 reflekteres det over hvordan alle sprintene gikk og hvordan de har blitt tatt med i planlegging av neste sprint. Der kommer det frem at backloggen ble iterert flere ganger i løpet av prosjektet, hvor krav som tok lenger tid en antatt ble delt opp i mindre oppgaver. Effektiviteten av arbeidet startet veldig bra, og viktige ting ble avklart og satt opp tidlig for å legge til rette for en god arbeidsflyt. Likevel dalte det en del midtveis hvor færre møter ble holdt, færre commits til GitHub og mindre å vise frem på hvert møte. Fordeling av roller skjedde først i sprint 1, mer om dette i avsnittet 4.1.3 om rollefordeling.

4.1.3 Rollefordeling

Underveis i sprint 1 startet fordelingen av roller, hvor Andreas ble valgt som produkteier og Marius som hoved-tester. Resten av teamet (inkludert Andreas og Marius) ble naturlig nok med i utviklingsteamet. For hver sprint ble det utnevnt en ny SCRUM Master, hvor Kjell-Magne med tidligere SCRUM erfaring ble valgt. Som produkteier hadde Andreas det overordnede ansvaret for backloggen, men delte ansvaret med Kjell-Magne tidlig i prosjektet for å bli kjent med administrering i Notion. Han har også stått ansvarlig for å kalle inn til møter, booke grupperom og lage Burndown chart til hver sprint. Marius har hatt mest kontakt med veileder og booket veiledningstimer. Han har i tillegg satt opp CI workflows for automatiske tester med Maven på GitHub og lokalt i IntelliJ.

Listen under viser oversikt over hvilke medlemmer som har vært SCRUM master under prosjektet. Sprint 0 var opprinnelig sprint 1, men siden den første sprinten offisielt skulle starte underveis i innlevering 2 endret den navnet til sprint 0. Siden sprintene ble organisert etter innleveringer rakk ikke alle medlemmene å forsøke seg i SCRUM master rollen.

Sprint nummer	SCRUM master
0	Kjell-Magne
1	Kjell-Magne
2	Arshad
3	Andreas

Tabell 4.1: Rollefordeling

4.2 Timelister

Siden dagen har det vært viktig å være transparente med å dokumentere arbeid utført av individuelle gruppemedlemmer. Det er utfordrende å måle arbeid kun i linjer kode pushet til GitHub, derfor tokk Andreas ansvaret for å sette opp en felles timeliste som alle har hatt tilgang til i OneDrive. Den var satt opp slik at hvert medlem fikk et eget ark å dokumentere sitt arbeid. I Hver rad skulle det noteres dato, antall timer og en kort kommentar til hva slags arbeid som mer utført. På toppen er det en celle som summerer antall timer. Et eget ark «total» oppsummerer alle timene utført av gruppa.

Gruppa måtte bli ekstra nøye i å dokumentere arbeidet etter det kom inn et nytt gruppemedlem. Derfor ble det opprettet en ny kolonne hvor det kunne dokumenteres hvilket aspekt av arbeid som ble utført, for eksempel programmering, design, Rapport og møte.

4.3 Versjonskontroll

For å kunne drive med utvikling ble det brukt versjonskontroll. Prosjektet bruker i hovedsak git for å kunne holde styr på endringer gjort i kode.

4.3.1 Github/Gitkraken

I starten av prosjektet ble det opprettet et Git repository på GitHub. Linken til repo'et finnes [her](#). Det ble opprettet brancher for at gruppemedlemmer kunne jobbe med forskjellig funksjonalitet på engang. Dette var praktisk for å unngå konflikter i kode. Hver gang en sprint var ferdig skulle alle brancher bli merget ned til main, men det ble i to tilfeller glemt og ikke gjort før etter sprint 2. GitKraken ble brukt som lokalt GUI verktøy fremfor GitHub desktop for en mer visuell fremstilling av brancher. Dette forenklet prosessene med å laste opp kode, eller å bytte branch for å jobbe med annen funksjonalitet. Et sikkerhetslag ble det lagt til godkjenning fra flere gruppemedlemmer ved en pull request til main. Det skjedde i et par tilfeller hvor noen valgte feil branch ved merging, men heldigvis var sikringen til stede så ingenting ble overskrevet.

4.3.2 Workflow i Git

Gruppen har forsøkt å følge GitHub flow som utgangspunkt for å arbeide med versjonskontroll. Dette er en branch-basert workflow, som har vært relativ enkel for medlemmene å sette seg inn i. Det starter i develop branchen, som grenes ut av main. Hvert medlem tar ansvar for en spesifikk oppgave, lager en ny branch for dette og navngir den noe relevant. Dersom et annet medlem ønsker å bidra på den samme oppgaven kan en opprette en ny branch utifra dette. Så fort funksjonalitet er testet og ansett som ferdig har koden blitt merget ned til sin opprinnelige branch. Til slutt merges det ned til develop og videre til main mot slutten av en sprint.

Denne flyten har ført til overraskende lite merge konflikts og gitt en behagelig introduksjon i å jobbe aktivt med brancher i Git.

4.4 Sprinter

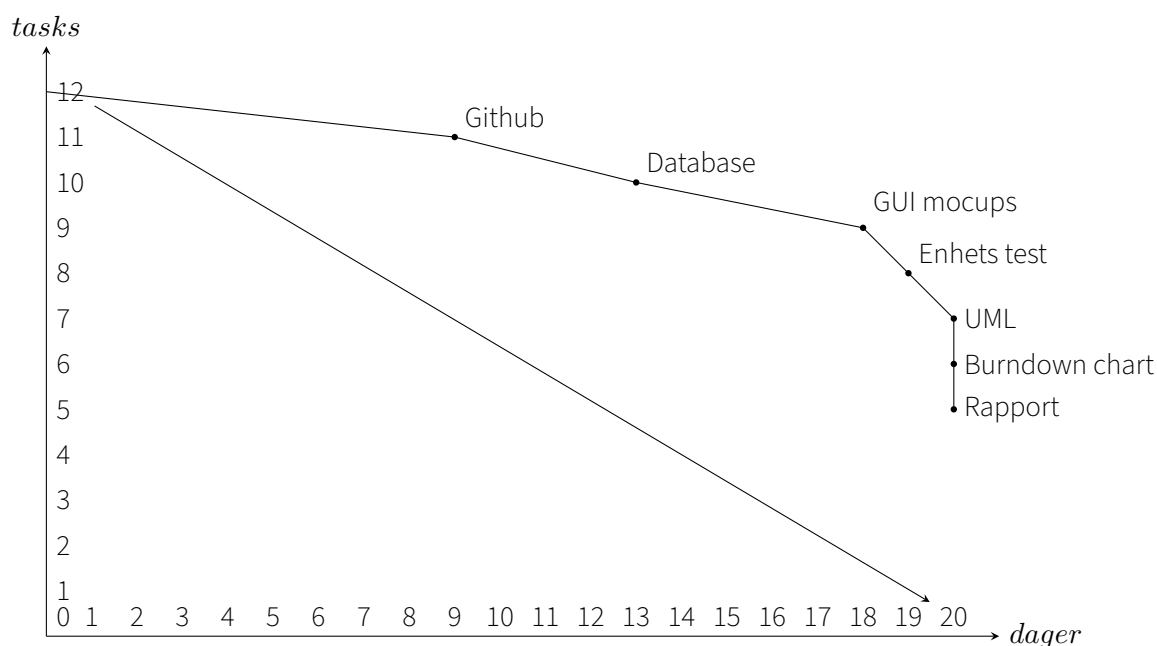
4.4.1 Sprint1 - retrospekt

Den første sprinten, sett bort i fra sprint 0 startet med høye ambisjoner, men må som forventet springe features ut i mindre oppgaver. Mine sider har blitt branchet ut underveis, og blir videre delt opp i sprint 2. Kompetanse test var estimert 3-5 dager og blir både branchet ut og får nye krav for å dekke målet om maks 1-2 dager per feature. Database oppsett er ansett som ferdig, men kan bli branchet ut i et nytt krav for dokumentdatabase.

Grappa innså å ha startet i feil rekkefølge med enkelte oppgaver. Implementering av klasser for IoT enheter og mine sider ble startet før design av diagrammer. Det har ført til misforståelser i blant annet mappestruktur og navngivning. Grappa forsøkte å dele arbeid best mulig mellom medlemmene, men andre fag hatt påvirkning på mengden arbeid som har blitt gjennomført. Derfor har programmeringen startet før klassediagrammer.

4.4.1.1 Burndown chart

Figur 4.1 viser burn-down chart for sprint 1, tilsvarende hele perioden for innlevering 1. Diagrammet viser de fleste oppgaver ble gjennomført nærme deadline, så det er stort forbedringspotensialet i å spre utførelsen av oppgaver bedre gjennom hele sprinten.



Figur 4.6: Burndown chart

4.4.1.2 Ferdige features

- Database oppsett - Anses som ferdig, men kan dukke opp nye krav som medfører justering eller opprettelse av nye tabeller og egenskaper.
- GitHub - Public repo med sikkerhetskrav og påkrevde branches. Alle medlemmer har fått tilgang.
- GUI sluttbruker mockups - Ikke behov for high fidelity prototype av produktet i denne omgang. Kun enkelte wireframes dersom det er nødvendig for kommunikasjonen i gruppa.

4.4.2 Sprint 2 - retrospekt

Siden innlevering 2 har trykket dalt noe og gruppen har følt å ikke strekke til for å fullføre features raskere enn backlogen vokser. Effektiviteten har tatt seg opp noe for å komme i mål med innlevering 3, men det legges ikke skjul på at faget til tider har blitt nedprioritert til fordel for fag med hyppigere innleveringer, deltidsjobber og unger. Til tross for at roller som produkt eier, SCRUM master og tester sklei ut har gruppen vedlikeholdt god dynamikk. Alle har vært flinke til å dokumentere arbeid i Notion og kommunisert over Discord utover fysiske møter på skolen.

Frem til nå har medlemmer valgt tilgjengelige oppgaver i Notion basert på kompetansenivå og interesser. Det har ført til at enkelte på har jobbet mer enn andre for å pushe prosjektet i riktig retning. For å sikre at alle får bidratt til testing og utvikling har IOT enhets klasser blitt branchet ut i seks forskjellige features som hver enkelt jobber med. Underveis i sprinten har det kommet et nytt medlem, mer om dette i kapittelet **Nytt gruppemedlem**

Mot slutten av sprinten har det pågått refaktoring og merging i GitHub for bedre struktur. Grunnet ufullstendig kode ble brancher verken merget til develop eller main ved slutføring av sprint 1. Det førte til unødvendig hindring av videre arbeid for å vente på merging av kritiske brancher. Opprydning i GitHub innebar å:

- Lage moduler for kjernekode, frontend, database og porter.
- Fjerne nye prosjekter som har blitt opprettet inne i prosjektet.
- Fjerne duplikater av main klassen med stor og liten forbokstav.
- Slå sammen kode ved å merge ned til develop for deretter å branche ut igjen.
- Finne ut hvorfor GitHub ignorerer gitignore filen.

4.4.2.1 Database

Etter forøk på uthenting og implementering av nye data i relasjonsdatabasen, ble det erfart at flere relasjoner i oppsettet ikke var optimale for en funksjonell løsning. Grunnet dette ble det gjort flere endringer på oppsettet av fremmed nøkler. Problemet var at relasjonene var i flere tilfeller satt opp feil vei, slik at foreldre tabeller ble oppfattet som barn av systemet.

Etter at disse relasjonene ble korrigert til ønsket oppsett, og relevant data ble oppdatert i riktig rekkefølge, fungerte løsningen som ønsket.

For en mer agil løsning vedrørende oppretting av nye bruker, ble det nødvendig og legge inn alle Norges postadresser i “sted” tabellen, slik at en ny kunde kunne opprettes med en norsk adresse uten at dette ble en konflikt mellom tabellene “sted” og “bruker”.

Det ble også lagt inn mer “troverdig” mock-data i “produkt” tabellen, slik at dette videre kan brukes for å opprette produkt artikler til sluttproduktet, med et ønsket GUI.

4.4.2.2 Nytt gruppemedlem

I midten av oktober tok gruppen imot et nytt medlem, Michael Tarvid fra en annen gruppe som ikke fungerte optimalt. Det ble gitt advarsler om en omfattende prosess å sette seg inn i en godt etablert gruppe så sent i prosjektet. Gruppen er kjent med at et nytt medlem forventer mer arbeid og potensielt utfordring med fordeling av oppgaver. Kjell-Magne tok på seg jobben som hovedkontakt i integrasjonsprosessen.

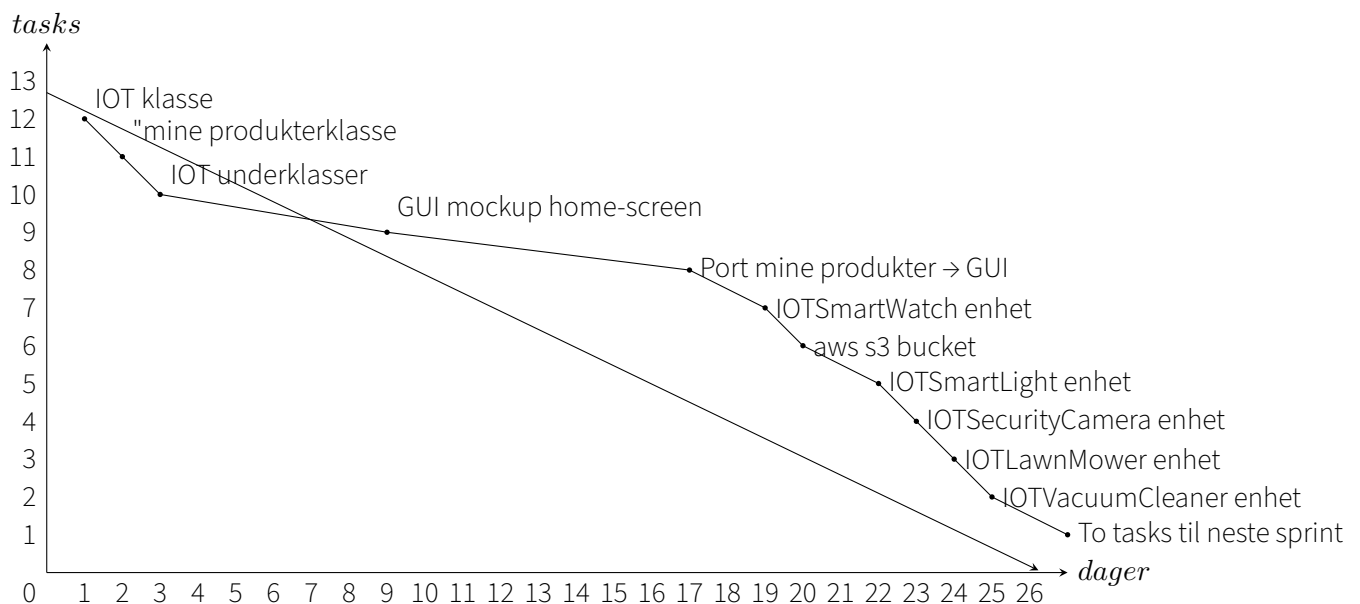
Integreringsprosessen startet med å lese tidligere innleverte rapporter, gå gjennom referater, dokumentasjon og commits gjort i GitHub. Som introduksjon til å bidra i utviklingen fikk han velge en IOT enhetsklasse å skrive og teste funksjonalitet.

Integrasjonsprosessen minnet om en effektiv «onboarding with the 5 C's».

- Klarhet (Clarity): Det nye medlemmet fikk innledningsvis informasjon om sin rolle i gruppen og prosjektet. Dette inkluderte en forklaring av egen utvelgelse av ansvarsområder og jobber gjennom notion, prosjektets mål og forventninger. Hovedkontakten satt seg også ned med det nye medlemmet og gikk gradvis gjennom arbeidet som er gjort og hvordan det er dokumentert i Notion.
- Samsvar (Compliance): Det nye medlemmet ble gjort kjent med nødvendig informasjon om relevante regler og prosedyrer som skulle følges for å sikre effektivt samarbeid.
- Kultur (Culture): Gruppen gjorde også en innsats for å gi det nye medlemmet innsikt i prosjektets kultur og struktur, samt timeplaner, møte tidspunkter og annen relevant informasjon.
- Tilknytning (Connection): Flere av gruppens medlemmer kjenner Michael personlig, noe som har ført til en mer enda mer effektiv introduksjon og kobling til gruppemedlemmene. Michael har også lagt inn en innsats i å sosialisere fysisk med flere av gruppemedlemmene.
- Oppfølging (Check-In): Til slutt omfattet integrasjonsprosessen oppfølgingsmøter med hovedkontakten til det nye medlemmet for å vurdere fremgangen, samle tilbakemeldinger og gjøre nødvendige justeringer i integreringsplanen. Disse møtene gjorde det mulig å adressere eventuelle utfordringer raskt og sikret at det nye medlemmet følte seg støttet gjennom hele overgangen.

“Overgangen til en ny gruppe har krevd en betydelig integrasjons prosess og opplevdes som veldig reell. Føles litt ut som om jeg har fått meg en IT jobb.” -Michael Tarvid

4.4.2.3 Burndown chart



4.4.3 Sprint 3 - retrospekt

Sprint 3 startet i en dalene periode med mindre arbeid og motivasjon, men tok seg veldig opp de siste 10-12 dagene før prosjektinnlevering. De høye ambisjonene hadde minsket, og med manglende kunnskap om kobling mellom database, kjerne og frontend, til hexagonal arkitektur førte til stagnering og en liten «kollaps». Etter hvert gikk det opp for flere på gruppa hvordan arkitekturen hang sammen, som førte til en «restart» av prosjektet. Det ble manglet fremdeles mye arbeid for en fungerende MVP og ideen om et fungerende GUI sto høyt på listen. Dermed ble det bestemt å dumme ned frontend til statiske HTML sider som får data ved hjelp av Javascript, fremfor rammeverk som Vue.js.

Av egen nysgjerrighet og interesse ble det slått fast at et fungerende REST API for å koble database og frontend skulle være oppnåelig, men ville kreve sene kvelder for å det til å virke. Det har også vært mye innsats i å få til en server som lytter og kommuniserer med en IOT enhet. Dette var så omfattende at løsningen ble implementert i terminal ved siden av GUI løsningen. Det ble også gjort omfattende arbeid i å få på plass CI løsning i GitHub som tillot å teste prosjektet på eksterne maskiner. Der kom det frem at MVPen hadde kritiske feil som ikke ble oppdaget når tester ble kjørt lokalt på egne maskiner.

Krav som kom på plass i sprint 3 basert på kanban boardet. Burndown chart ble det ikke tid til å sette opp før levering.

- Login side
- Ny brukerprofil

- Kompetanse test, kun backend og ingen implementasjon
- Produkt artikler/katalog
- Cookies/Session lagring

Integreringen av Michael har ikke vært gunstig og ført til at han har falt utenfor en del av utviklingen og heller skriver mest på rapporten. Det skyldes nok i all hovedsak at han kom inn i gruppa på et så sent tidspunkt at det har vært utfordrende å sette seg inn i den tekniske delen av prosjektet. Totalt sett har Sprint 3 vært den mest effektive for utvikling og implementasjon av MVPen har stort sett vært utviklet i denne perioden.

4.5 Diskusjon rundt erfaringer

Grunnet at medlemmene i gruppa har i utgangspunktet sittet på ulike kunnskapsnivåer gjort at innsikt i MVP er spredt litt utover i prosjektet. Gruppa har vært tydelig på at alle kan skal kunne jobbe med det er gode på og trives med, men må også gå litt ut av komfortsonen for å lære mer om prosjektet i alle ledd. Samtidig har de som er gode på programmering giret ned et par hakk for å tillate alle på gruppa i å utvikle for blant annet hexagonal arkitektur.

Til tross for dette har de med best innsikt i utvikling og arkitektur forsøkt etter beste evne å introdusere andre i konseptene som ligger bak, utover hva som er nevnt i forelesning. Det foreligger likevel stor differanse i hvem som har gjort hva, for eksempel står Marius for den tyngre bakenforliggende programmeringen, oppsett i GitHub og drivkraft bak arkitekturen. Andreas har hatt hovedansvar for databaser. Kjell-Magne har stått for design, utforming og senere på implementasjon av APIer. Arshad har hatt mest kunnskap om Latex og rapportskrivning, og vært pådriver for utvikling av frontend og satt først opp koblingen mellom database og kjernen.

Løsning

5.1 Github repo

Det ble opprettet et Github repository. Den kan aksesseres her: [Github repository](#)

5.2 Avhengigheter

5.2.1 Java 21

Prosjektet bruker Java som primærspåk for å skrive kjernefunksjonalitet og er satt opp i JDK 21.0.2. Java 21 LTS vil leve i mange år og støttes av andre avhengigheter som Mockito, som foreløpig ikke støttes av Java 23. Maven benyttes som byggeverktøy. Det sto mellom å skrive kjernekode i Java, C# eller Python, og ble gjort en votering i gruppa hvor Java fikk såvidt flere stemmer enn C#. Begrunnelsen bygger på at Java undervises i kurset og nylig tilnærming i kunnskap om OOP i Java gjennom ITF10619 programmering 2.

5.2.2 JUnit 5 og Mockito

Benytter Java sitt eget rammeverk for testing, JUnit til å skrive tester. Rammeverket Mockito brukes i sammenheng med JUnit for å effektivisere testing av funksjonalitet, som å lage falske objekter av klasser uten å være avhengig av eksterne løsninger. JUnit og Mockito er begge gjennomgående temaer i kurset.

5.2.2.1 JSONassert/Unirest

JSONassert er et Java-bibliotek som forenkler sammenligning av JSON-strukturer i tester, og gir mulighet for å validere JSON-data ved å bruke enkle uttrykk, med detaljerte feilmeldinger ved uoverensstemmelser. Unirest er et HTTP-klientbibliotek som gjør det enkelt å sende HTTP-forespørsler som GET, POST, PUT, og DELETE i Java, og håndtere svarene med minimal kode. Begge verktøyene bidrar til å forbedre testing og API-kommunikasjon i Java-prosjekter.

5.2.3 Jackson

Jackson^[2] er et populært rammeverk for data-binding i Java, hovedsakelig brukes til å konvertere mellom Java-objekter til JSON-format og omvendt. Det gjør det enkelt å serialisere data (fra JavaScript Object Notation). Det er svært fleksibelt og kraftig, og kan håndtere både serialisering (konvertere Java-objekter til JSON-streng) og deserialisere data (fra JSON-streng til objekt), noe som er nyttig når man jobber med API-er eller andre systemer som utveksler JSON-data.

Jackson ble valgt nemlig for å kunne serialisere objekter til JSON, slik at de kan returneres via API endepunkt videre til frontend. Dette er en klar fordel for da slipper man bruke tid på å skrive disse JSON objektene selv. Det er også mulig å bruke Gson, som gjør det til et populært valg for JSON-håndtering, og en av de potensielle ulempene er i implementasjonen. Gson kan virke litt enklere å bruke i visse sammenhenger, men Jackson ble allikevel valgt siden den utfyller samme funksjonalitet som trengs i Java-prosjektet.

5.2.4 Javalin

Javalin[3] er et lettvekts web-rammeverk for Java og Kotlin, designet for å forenkle utviklingen av webapplikasjoner og API-er. Det er kjent for sin enkelhet, minimalisme og fleksibilitet. I motsetning til andre rammeverk som Spring Boot, har ikke Javalin mange avhengigheter eller kompliserte oppsett. Noe som gjør det enkelt å komme i gang. Javalin støtter RESTful API-er, sanntidskommunikasjon med WebSockets, og enkel ruting for HTTP-metoder som GET, POST, PUT og DELETE.

Javalin ble foreslått brukt til å lage et API fordi det ble undervist i gjennom programmering 2 faget, og er noe som det finnes nok dokumentasjon og hjelp til på nett. Det foreligger kildekode og videoer fra HiØ på nett som kan brukes til hjelp, som videre gir en stor fordel i prosjektet. Det er heller ikke et stort eller komplekst rammeverk. Ulempen ved å bruke dne er nettopp at den er et lettvekts rammeverk, så noe funksjonalitet kan mangle. Slik det er nå så virker det som at CRUD funksjonalitet er støttet, men tiden vil vise om noe mangler eller hvis Javalin ikke holder mål.

5.2.5 MySQL JDBC driver

For å kunne kommunisere med databasen har det blitt brukt en MySQL JDBC driver. Denne gjør det mulig for applikasjonen å kunne kjøre spørringer mot databasen. Tanken er å kunne bruke denne for å håndtere spørringer på et lavest mulig nivå, og slippe å bruke større rammeverk for dette.

5.2.6 Azure MySQL server

Relasjonsdatabase i Microsoft Azure benyttes til å lagre data om applikasjonen. Fordelene er at alle på gruppa får tilgang til å teste funksjonalitet og gir mer tilgang til å administrere databasen enn for eksempel å benytte skolens itstud server. Ulempen er at vi har mindre kontroll på sikkerheten og potensielt låser seg til Microsoft sin plattform.

5.2.7 AWS S3 bucket

Relasjonsdatabasen egnet seg dårlig til å håndtere multimedia data. Da oppsto det et behov for en database til å håndtere dette i sluttproduktet. Denne problemstillingen ble tatt opp i plenum med gruppen, og etter å ha vurdert forskjellige alternativer, ble det til slutt enighet om at en god løsning ville være "AWS S3" sin database tjeneste

Det ble opprettet en konto på Amazon sine tjenester, som videre ble brukt til å opprette en "public bucket", som skulle brukes til å holde på bilder og eventuelt andre mediefiler til sluttproduktet.

5.2.8 JBCrypt

JBCrypt er et Java-bibliotek som brukes for å kryptere og verifisere passord ved hjelp av bcrypt-algoritmen. Bcrypt er en sikker hashingalgoritme designet for å gjøre det vanskeligere å knekke passord ved å bruke en tidskompleksitetsfaktor, som gjør at hashingen blir mer ressurskrevende etter hvert som datamaskiner blir raskere. JBCrypt gir en enkel måte å implementere passordbeskyttelse på, og brukes for lagring av sensitive data som passord i applikasjoner. I dette prosjektet ble det brukt til sikker lagring av passord i databasen

5.3 Klasser

For å komme igang med prosjektet ble det opprettet noen løse klasser. Det er overordnet klasse IOTDevice, deretter arver alle andre fra den. Koden til IOTDevice blir vist her i rapporten, mens resten av klassene kan inspiseres i git repoet.

5.4 Database

Database oppsett

Relasjonsdatabase ble valgt for å lagre generell data om produktet siden datamengden er rimelig begrenset for prosjektet. Prosjektgruppen har en generell forståelse av hvordan strukturen kan fungere, men at ved større datamengder vil bli vurdert å implementere datalagringsløsningen på en NoSQL dokumentdatabase.

Databasen er satt opp i Microsoft Azure sin MySQL database slik at alle til enhver tid kan utføre testing på sine ansvarsområder. En logisk løsning for databasens arkitektur ble tatt opp i plenum før tabellene kunne settes opp. Se figur 3.1 for EER-diagram av forslag til databasens struktur.

Tabeller ble opprettet og ChatGPT ble benyttet til å generere dummy-data for å fylle opp relasjoner. Det ble så utført diverse spørringer mot systemer, slik at det kunne konkluderes at databasens struktur innfridde til forventningene.

5.5 Kobling mot database

For å sørge for at klassene kan kommunisere med databasen ble det brukt en JDBC løsning. Tanken var å komme med en enkelt enhet og eksempelkode som fungerte, og at det skulle være så lavnivå som overhodet mulig. I og med at et av gruppemedlemmene har jobbet med dette før og hadde kode liggende, så ble det mye kopiering og justering for å få koden til å fungere i vårt eksempel.

5.5.1 Mysql util

Det ble opprettet en util klasse som inneholder databaseinformasjonen og oppretter tilkoblinger. Denne blir kalt for å opprette en kobling til databasen og kjøre SQL spørringer.

5.5.2 SQL spørringene

SQL spørringene ble gjort ved hjelp av PreparedStatement og ExecuteQuery fra JDBC.

5.6 API endepunkt

For å kunne servere frontenden med data, så ble det opprettet forskjellige API endepunkter ved hjelp av Javalin. Disse endepunktene var for det meste GET endepunkter, hvor det enten ble servert en frontend fil, eller så ble det servert data om brukere/produkter i form av JSON. Dette API'et hadde også mulighet for å kunne ta imot en POST request fra login. Disse funksjonene ble lagt inn i en egen fil kalt apiAdapter. Senere ble det laget flere API og mer funksjonalitet, og det opprinnelige adapteret ble erstattet med flere andre API. Det ble laget totalt fire API filer; en for autentisering, en for å hente ut bruker(e), en for å hente ut produkt(er), og en for å opprette ny bruker i databasen.

5.7 Webløsning

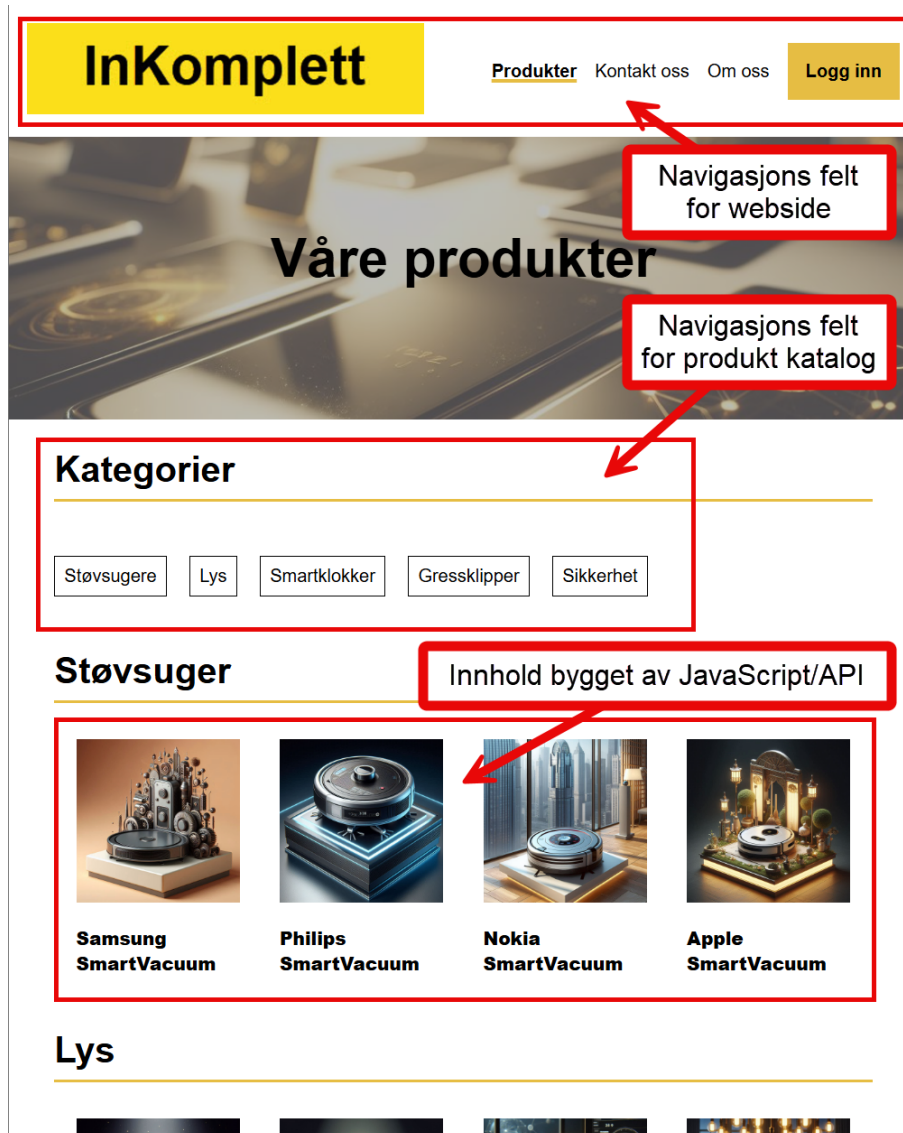
I dette prosjektet ble det laget en enkelt frontend. Først ble det satset på en Vue frontend, men det var noen rare feilmeldinger som oppstod, og det viste seg at ikke alle nettlesere fungerte like bra med den. Så da ble det bestemt å dumme det ned vesentlig, og heller servere frontend filene statisk som html filer. Deler av det grafiske grensesnittet er tatt fra en eksamensoppgave^[4]

5.7.1 Produktkatalog

Produktkatalogen fungerer som en startside for produktet. Den er bygget opp med et navigasjonsfelt som er standardisert for alle sider, slik at det skal være lett å finne frem til de ønskede områdene på tjenesten. Under navigasjonsfeltet vil bruker bli introdusert for selve produktkatalogen.

Her vil bruker få en enkel oversikt over de enheter som støttes av leverandøren. Disse vil være kategorisert, slik at det er enkelt for brukeren å navigere seg gjennom utvalget. Det er også lagt inn en navigeringsseksjon over produktene med de kategoriene som nå finnes i katalogen, som tar brukeren til den gitte kategorien ved å klikke på en av grupperingene. Om brukeren klikker på et produkt, vil de bli tatt til den relevante produktartikkelen.

Siden er satt opp med en dynamisk løsning ved bruk av JavaScript og API-funksjonalitet. Dette innebærer at katalogen blir bygget opp av data som hentes fra databasesystemet. På denne måten vil det være svært enkelt for leverandøren å oppdatere katalogen ved å legge til eller fjerne produkter direkte fra databasen uten å måtte gjøre spesifikke endringer på selve nettsiden.



Figur 5.1: Startside webløsning

5.7.1.1 Fremtidige Utviklingsprioriteringer

I videre utvikling av produktkatalogen vil det bli prioritert å implementere en funksjon for innloggede brukere som gir en liste over anbefalte enheter. Disse anbefalingene vil være basert på de IOT enhetene brukeren allerede har registrert i sin brukerprofil. Hensikten med denne funksjonen er todelt: for det første vil det forbedre brukeropplevelsen ved å gi personaliserte forslag som er relevante for brukeren, og for det andre vil det kunne bidra til å generere økt potensiale for salg av produkter.

Den foreslåtte funksjonaliteten vil kreve en algoritme for produktanbefalinger, som integrerer data fra brukerens produktregistreringer med relevante produktkategorier i katalogen. En slik løsning vil ikke bare styrke brukervennligheten, men også åpne for nye muligheter innen målrettet markedsføring og økt kundelojalitet. For å realisere dette er det viktig å sikre sømløs integrasjon mellom brukerens

data og katalogens innhold, samtidig som personvern og datasikkerhet ivaretas i tråd med gjeldende lover og forskrifter.

5.7.2 Artikler

Når bruker klikker på et produkt via katalogen vil de bli tatt til produkt artikkelen for det gitte produktet. For å gjøre ting så konsist som mulig for bruker, har denne siden tilsvarende struktur som produktkatalogen, men med fokus på det gitte produktet. Her kommer det frem en tittel på selve produktet med en liten navigasjons løsning som tar bruker til forskjellige seksjoner av artikkelen. Det kommer også frem et forstørret bilde av produktet, slik at bruker lett skal kunne se at de er inne på riktig artikkel.

Over bildet av selve produktet finnes det også en knapp med teksten “Se produktet i nettbutikken”. Da det ikke finnes noen nettbutikk per i dag, så leder denne linken kun tilbake til produktkatalogen, men om det hadde foreligget en faktisk nettbutikk, så ville denne linken ha tatt bruker direkte til produktets salgsside.

Tilsvarende som produktkatalogen, så blir også artikler generert av data fra database løsningen via JavaScript og API funksjonalitet. Dette vil si at det vil være enkelt for leverandør og editere enheters innhold og avhengigheter via databasen uten å måtte forholde seg til struktur på webtjenesten.

Per dags dato foreligger det kun konkret informasjon om selve produktet, samt hvilke avhengigheter dette produktet har for funksjonalitet. Ved videre utvikling av denne løsningen ville det bli prioritert å legge inn funksjonalitet for å direkte installere dette produktet for innloggede brukere.

Avhengigheter som kommer frem på produktet er også kun ment for illustrasjon og er dermed satt opp mot tilfeldige produkter som finnes i databasen. Ved videre utvikling av dette produktet ville det bli lagt inn enheter med genuin avhengighet, slik som rutere og tilsvarende enheter.

Det er også verdt å nevne at det er behov for videre utvikling av API funksjonalitet vedrørende avhengigheter, da det i dag bare blir hentet ut en avhengighet selv om et produkt potensielt har flere. Her er alt klart satt opp i webløsningen via Javascript, så det er kun API løsningen som har behov for en liten korreks her, slik at all relevant data kommer med fremfor kun ett enkelt element.

InKomplett[Produkter](#)[Kontakt oss](#)[Om oss](#)[Logg inn](#)

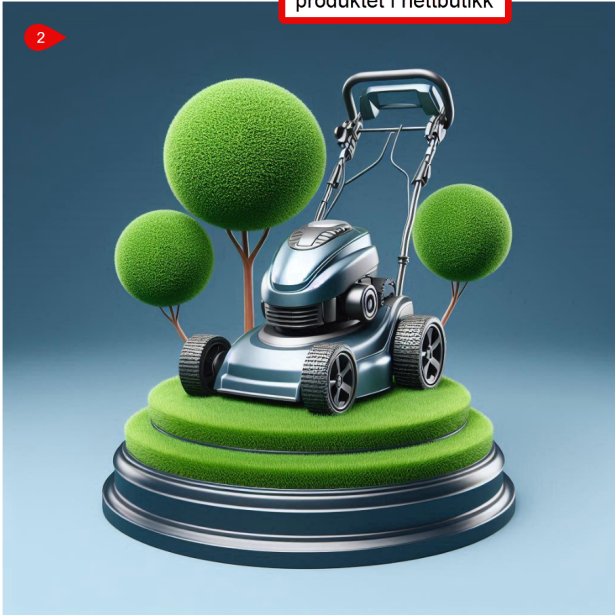
Samsung SmartMower ¹

[Om produktet](#)[Spesifikasjoner](#)[Avhengigheter](#)

Navigasjonsfelt for produkt artikkel.

[Se produktet i nettbutikken](#)

Knapp tar bruker til produktet i nettbutikk

²

Om produktet

Roboterklipper med GPS-navigasjon og app-kontroll. ³

Spesifikasjoner

- Kategori: Gardening
- Modell: Mower Pro
- Garanti: 36 måneder
- EAN: S9876543210123

⁴

Punkt 1, 2, 3, 4 og 5 genereres av data fra databasen ved bruk av JavaScript/API implementasjon.

Avhengigheter

⁵

Apple SmartMower

Figur 5.2: Enkelt produkt

5.7.3 Login funksjonalitet

Det ble utviklet en login funksjonalitet i applikasjonen. Det er et eget skjema som tar imot brukernavn og passord, og som kjører en POST forespørsel mot et gitt endepunkt. Endepunktet tar så imot dataene og kjører en SQL spørring for å kunne sjekke hvorvidt brukernavn og passord stemmer. En viktig ting å bemerke seg er at det blir brukt en avhengighet til å håndtere kryptering og dekryptering av passord.

Hvis det viser seg at login er vellykket vil frontend javascript så videresende bruker til hjemmesiden for brukeren, hvor det vil bli vist informasjon om brukeren. Hvis det oppstår feilmeldinger vil det vises like under skjemaet for innlogging.

Det vil også bli lagret en cookie lokalt. Denne kan brukes i funksjonalitet for å sikre at visse endepunkt og områder i webapplikasjonen kan sikres mot at uvedkommende får tilgang.

Det er også laget en logg ut funksjonalitet hvor brukeren vil bli logget ut, og eventuelle cookies som er lagret vil bli fjernet.

The screenshot shows the login page for 'InKomplett'. At the top, there is a yellow header with the 'InKomplett' logo on the left and navigation links 'Produkter', 'Kontakt oss', 'Om oss', and a 'Logg inn' button on the right. Below the header, the 'Logg inn' section contains two input fields for 'Epost:' and 'Passord:'. Under these fields are two buttons: 'Logg inn' (blue) and 'Ny Bruker' (blue). Below the buttons, the 'Login detaljer' section lists two example users: 'Epost: john.doe@example.com' with 'Passord: password123', and 'Epost: jane.smith@example.com' with 'Passord: password456'. A red box with a red arrow points to the first example, containing the text 'Eksisterende brukere for å teste innlogging.'

Figur 5.3: Login side

5.7.4 Legge til ny bruker

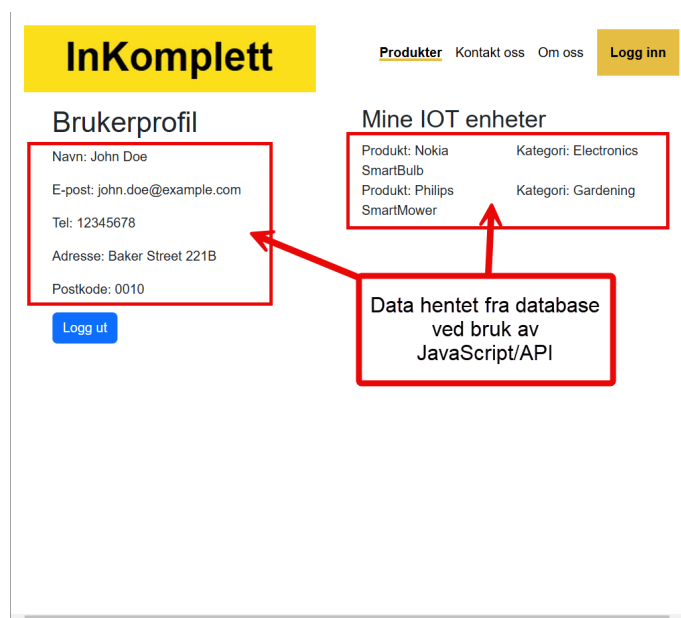
Når login funksjonalitet var på plass ble enkelte ivrige og ville ha registrering også. Løsningen ble et skjema som tar imot personlig info som navn, telefonnummer, bosted, e-post og passord. Dataen sendes med en POST metode til et gitt endepunkt som tar imot og behandler dataen. Det kjøres først en test for å sjekke om noen av feltene er tomme, for deretter å kjøre en SQL spørring mot databasen for å se om en bruker med tilsvarende e-post adresse eksisterer. Dersom ingen av sjekkene inntreffer blir ønsket passord kryptert med BCrypt, etterfulgt av funksjonalitet som forebygger SQL injection, for til slutt en SQL spørring som setter inn den nye brukeren i databasen. Bruker får en feilmelding skrevet ut på siden dersom en av sjekkene feiler, dessverre er de ganske vagt implementert og bruker får kun beskjed at det har skjedd en feil hvis e-post adressen eksisterer fra før.

5.7.5 Hvordan IOT enheter blir hentet ut

IOT enheter kan bli hentet ut fra databasen på flere måter, hovedsakelig med GET metoder til et endepunkt i JSON. Felles er at det kjøres en SQL spørring som tar inn ønsket data fra enhetene og sender videre til APllet. Det er flere endepunkter ved å oppgi parametere kan filtrere på lister med enheter. For eksempel er det mulig å hente ut alle enheter, men også en enkelt enhet ved å spesifisere ID. Det er også mulig å hente ut en liste med produkter eid av en bruker når bruker ID er nevnt, som vises i avsnitt 5.7.6.

5.7.6 Personlig side etter login

Etter at man har logget inn så vil det vises informasjon om brukeren og eventuelle IOT enheter som er knyttet til brukeren. Disse vil lastes ved hjelp av Javascript som vist på bildet under:



Figur 5.4: Brukerside logget in

Testing

6.1 CI Integrasjon

Det er satt opp kontinuerlig integrasjon (CI) på dette prosjektet ved bruk av GitHub Actions sin Maven workflow. Denne starter opp en ekstern maskin som bygger og kjører testene i programmet. CI kan også avdekke problemer med avhengigheter, bygging og kjøring. Vi har konfigurert dette med en YAML-fil hvor vi har endret hvordan stegene blir skrevet ut til oss, samt hvilke brancher og når den skal kjøres. Vi har satt opp CI for develop- og main-branchene, slik at den kjører når man merger, pusher eller starter en pull request til disse branchene.

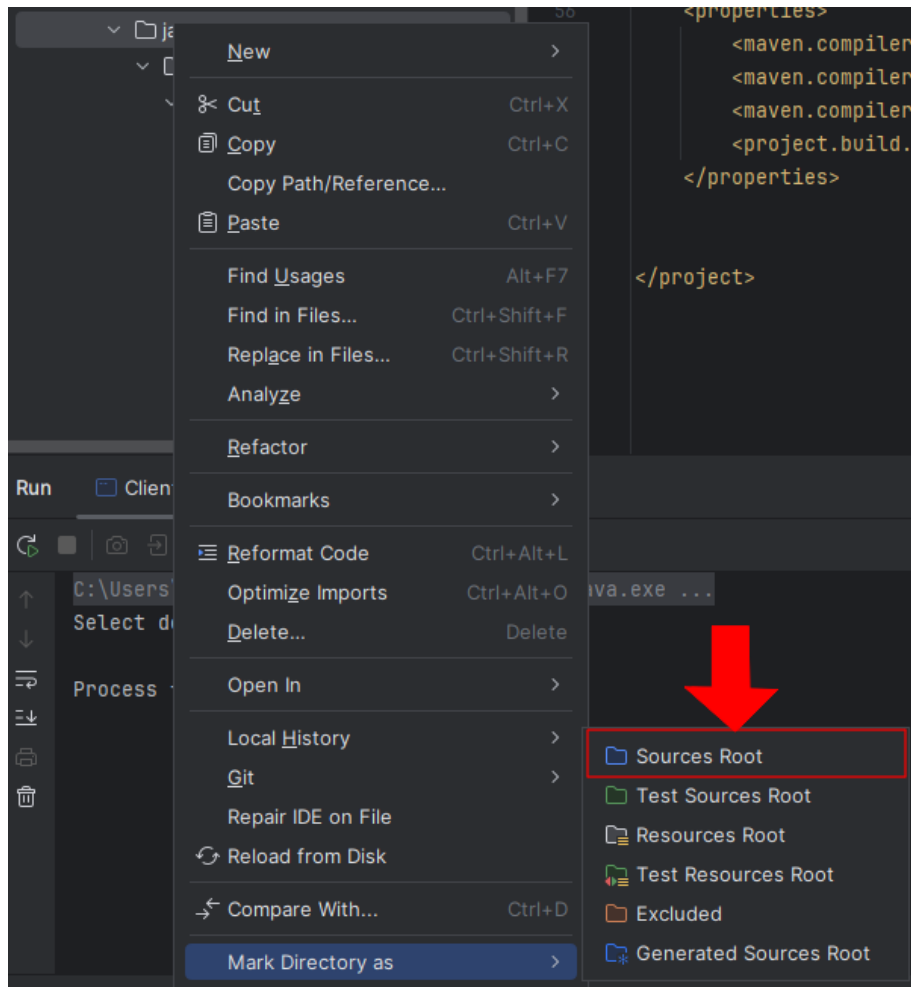
For at dette skal fungere optimalt, bruker vi Maven Surefire-pluginen for å generere testrapporter som vises når CI kjører. Dette har vist seg å være et veldig nyttig verktøy, da det avdekket flere problemer med både noen tester og hvordan rotmappen på prosjektet vårt ikke var riktig konfigurert. Det krevde mye arbeid å rette opp i feilene, da vi la til CI veldig sent i prosjektet. Hadde vi implementert CI tidligere, ville vi ha oppdaget problemene når de oppstod og kunne ha fikset dem fortløpende. CI bidrar også til å redusere antall slurvefeil og fungerer som et ekstra lag med kvalitetssikring når vi pusher til main, i tillegg til regelsettene vi har satt opp for main-branchen.

6.2 Integrasjonstest

Hvordan kjøre manuel integrasjons test mellom klient (FalskIoTEnhet) server

Det har oppstått problemer med merging mellom branch som inneholder server, klient. Dette har gjort at vi har merget ned resten av prosjektet utenom server, klient til main. For å kunne kjøre testen må man derfor åpne prosjektet som ligger i Test_module.zip.

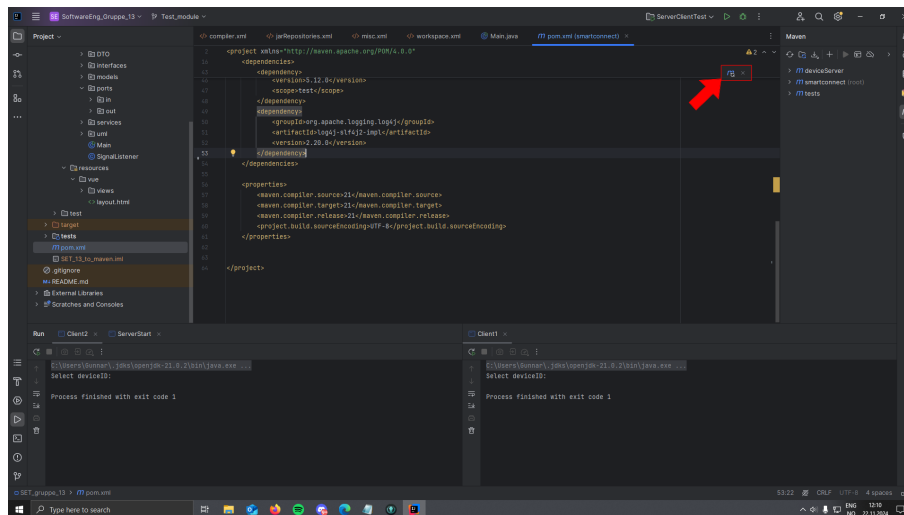
Hvis java mappen under src/main ikke er blå har ikke intellij gjenkjent mappen som root mappe, Høyre klikk på java mappen og velg mark directory for så å velge root mappe.



Figur 6.1: Sette root mappe

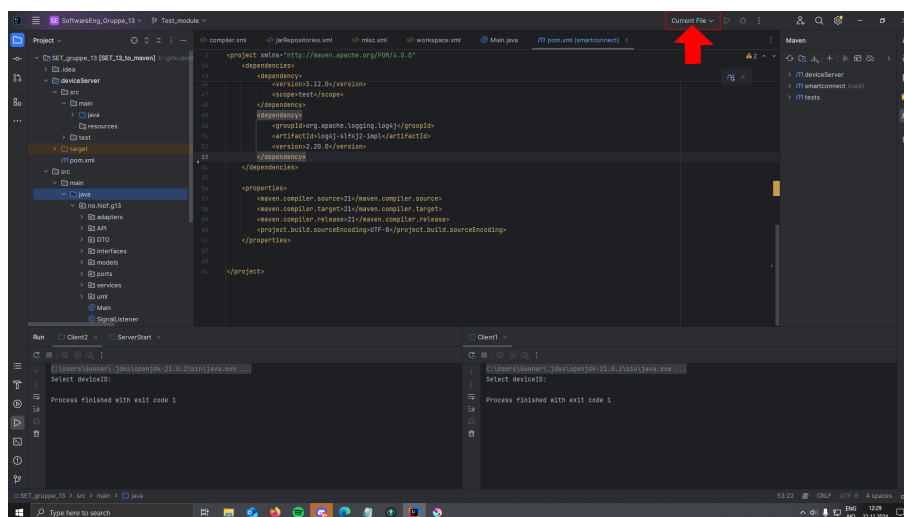
Steg 1: Åpne prosjektet i IntelliJ.

Steg 2: Hvis maven laster inn avhengigheter automatisk hopp til steg 3. Hvis ikke gå til pom.xml rett under SET_gruppe_13 og trykk på last inn maven endringer.



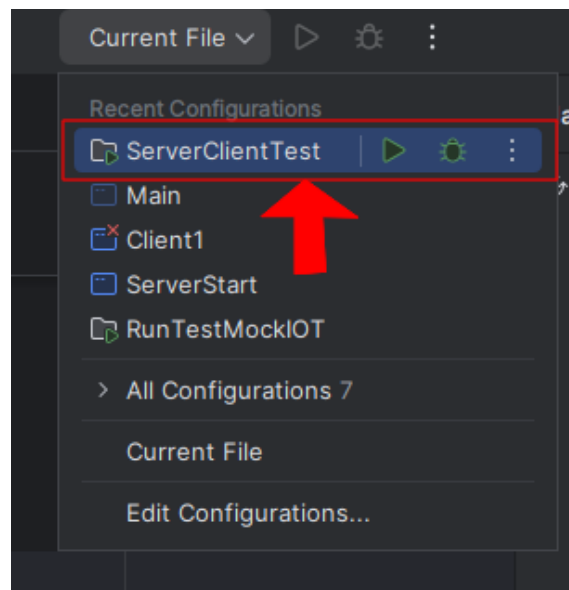
Figur 6.2: Laste pom.xml

Steg 3: trykk på nedtrekksmenyen i øvre høyre hjørne



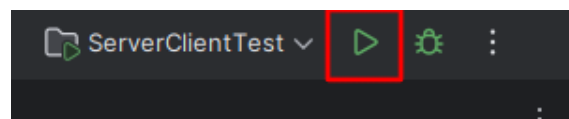
Figur 6.3: Compound

Steg 4: Velg ServerClientTest



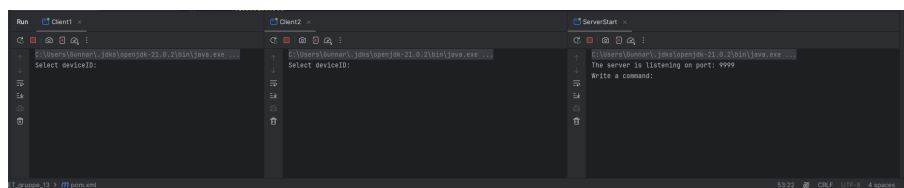
Figur 6.4: Server og klient test

Steg 5: trykk på kjør knappen eller shift+f10



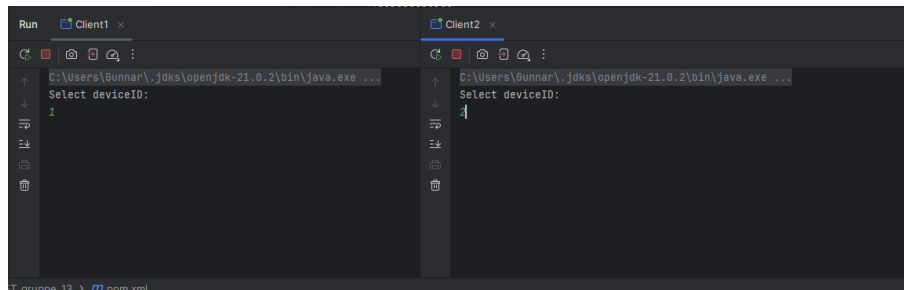
Figur 6.5: Trykk play

Steg 6: Flytt på de 3 terminal vinduene som starter slik at du kan se alle 3 på en og samme tid



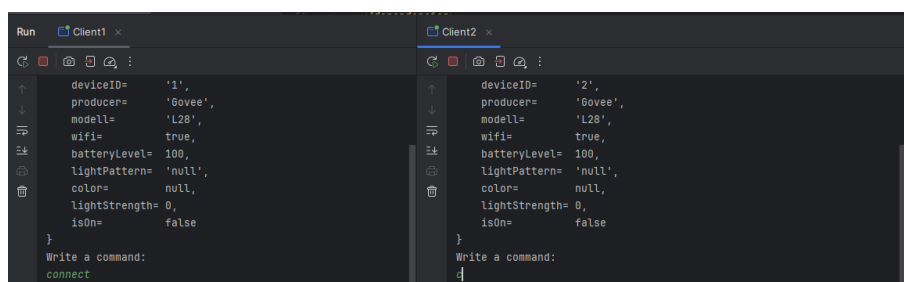
Figur 6.6: Terminalvinduer

Steg 7: Velg en enhetsID på de to terminalvinduerne Client1 og Client2, Dette kan være hvilken som helst tekst streng. Hvis du velger ID 1, 2, 3 eller 5 vil det finnes en IOTEnhet med den ID hos brukeren og du kan teste det å oppdatere lys instillingene på Klienten fra IOTEnheten i kjernen med samme ID.



Figur 6.7: Skrive inn klientID i terminal

Steg 8: Skriv in connect eller bokstaven c på Client1 og Client2. Dette vil koble klientene opp mot serveren.



Figur 6.8: Koble opp mot serveren

Steg 9: Prøv deg frem med forskjellige kommandoer både på Client1, Client2 og ServerStart.

Kommandoer for klientene (Må skrives i terminal med navn Client1, Client2):

c eller connect:

Kobler klienten opp mot serveren.

d eller disconnect:

Kobler klienten fra serveren.

s eller status:

Skriver ut om enheten fortsatt er tilkoblet og hva slags innstillinger som er satt på klienten.

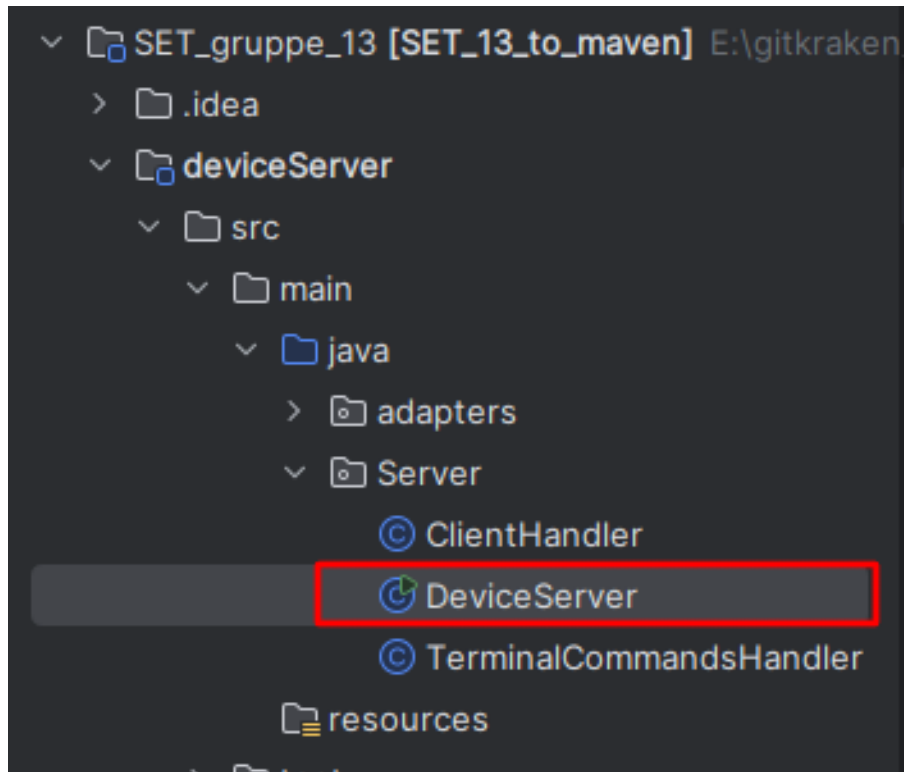
e eller exit:

Kobler klienten fra og slår av klienten.

sp eller setPort:

Denne setter hvilken port som klienten skal koble seg på. Denne kommandoen skal KUN brukes hvis klienten ikke klarer å koble seg opp mot serveren, fordi en annen enhet har tatt i bruk porten

serveren skal startes med. Dette krever at du først går inn i klassen DeviceServer og endrer ServerSocket(9999) til ServerSocket(0) (Dette gjør at operativsystemet velger en port for serveren). Når du da starter serveren skriver den ut porten den kjører på og du må da bruke denne kommandoen og sette porten til å være det samme som serveren kjører på før du så bruker kommandoen connect.



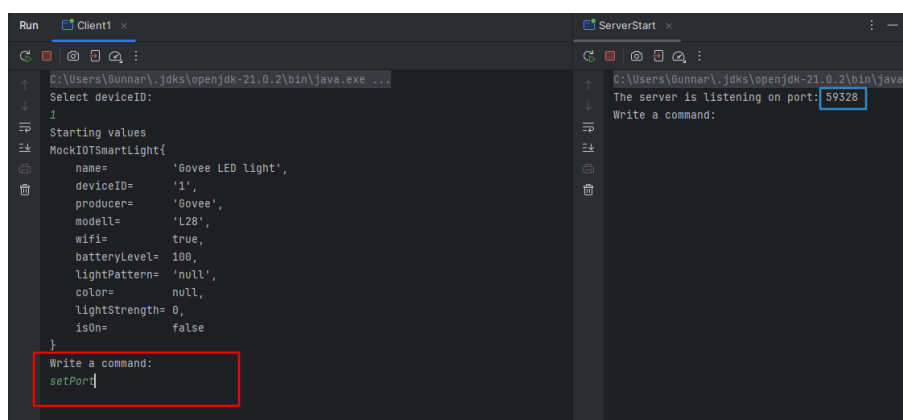
Figur 6.9: DeviceServer klassen

```
/**
 * Starts the server by generating a server object and uses the startServer() to make the server ready to listen
 * for connections.
 * It also starts the TerminalCommandsHandler witch handles the terminal commands.
 * @param args
 * @throws IOException
 */
public static void main(String[] args) throws IOException {
    //MaRoelIT
    /*There have been instances where the port has been taken by other programs, if that happens change it from
    new ServerSocket(9999); to new ServerSocket(0); witch will let the os select a free port for you.
    You must then also use the command setPort on the client side to match the port on the server, this will be
    printed out in the terminal window of the server.
    */
    ServerSocket serverSocket = new ServerSocket(port: 9999);
    DeviceServer server = new DeviceServer(serverSocket);
    TerminalCommandsHandler terminalCommandsHandler = new TerminalCommandsHandler(server);
    Thread thread = new Thread(terminalCommandsHandler);
    thread.start();
    server.startServer();
}
```

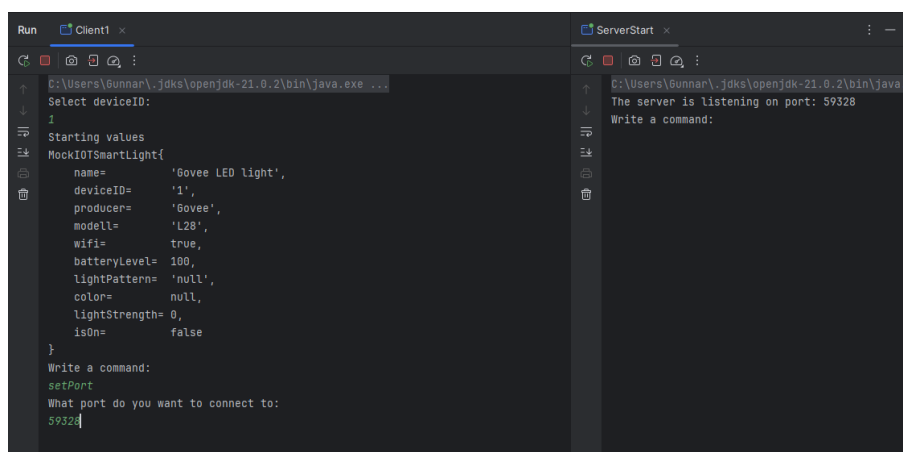
Figur 6.10: ServerSocket(9999)

```
/**
 * Starts the server by generating a server object and uses the startServer() to make the server ready to listen
 * for connections.
 * It also starts the TerminalCommandsHandler witch handles the terminal commands.
 * @param args
 * @throws IOException
 */
public static void main(String[] args) throws IOException {
    /*MaRoelIT*/
    /*There have been instances where the port has been taken by other programs, if that happens change it from
    new ServerSocket(9999); to new ServerSocket(0); witch will let the os select a free port for you.
    You must then also use the command setPort on the client side to match the port on the server, this will be
    printed out in the terminal window of the server*/
    ServerSocket serverSocket = new ServerSocket(port: 0);
    DeviceServer server = new DeviceServer(serverSocket);
    TerminalCommandsHandler terminalCommandsHandler = new TerminalCommandsHandler(server);
    Thread thread = new Thread(terminalCommandsHandler);
    thread.start();
    server.startServer();
}
```

Figur 6.11: ServerSocket(0)



Figur 6.12: Sette port med kommando - 1



Figur 6.13: Sette port med kommando - 2

Kommandoer for Server (Må skrives i terminal med navn ServerStart):

g eller getDeviceIDs:

Skriver ut alle enhetene som er koblet til serveren med enhets ID deres.

s eller send:

sender en tekst streng til en klient; du vil bli spurt om hva du vil sende og da er det bare å skrive en eller annen form for tekst. Så blir du spurt om hvilken enhet du vil sende det til som du velger med å skrive inn enhets ID.

cl eller changeLight:

Manuell endring av lysinnstillinger på en Client1; du blir bedt om å gi en enhets ID du ønsker å sende til, før du blir spurt om lys pattern, farge og lys styrke. så blir det sendt over til klienten, og så overskriver den lys innstillingene til klienten.

on eller turnOn:

Setter IOTEnhetens on verdi til true (Skal simulere at enheten er slått på)

off eller turnOff:

Setter IOTEnhetens on verdi til false (Skal simulere at enheten er slått av)

lc eller lightSettingsFromCore:

Leter fram enheten i kjernen med samme enhets ID som klienten, og endrer klientens lys innstillinger til å samsvare med kjernens IOTEnhet. (Dette skal simulere det å oppdatere innstillingene til enheten når den slås på, etter å ha vært avslått. Dette oppdateres mens bruker har endret innstillinger i applikasjonen).

6.3 Enhetstest

I denne rapporten blir kun et eksempel på hvordan testing ble gjort vist. For denne rapporten vises testing av klassen IOTSmartWatch, resten av testene kan finnes på github repositoriet.

Klassen IOTSmartWatch ble opprettet med hensyn til TDD fremgang.

Etter at klassens fundamentale struktur var opprettet med hensyn på arv fra forelder-klasser, ble videre funksjoner opprettet parallelt med en test-klasse, som ble kontinuerlig testet for hver funksjon opprettet i klassen.

I testmiljøet blir det opprettet et mock-objekt basert på klassens konstruktør, som testene bruker til å teste funksjonene.

```
@BeforeEach
void setUp() {
    ArrayList<String> funksjoner = new ArrayList<>();
    funksjoner.add("Puls Monitor");
    funksjoner.add("GPS");

    HashMap<String, Integer> size = new HashMap<>();
    size.put("width", 40);
}
```

```

        size.put("height", 50);

        mockSmartWatch = new IOTSmartWatch(
            "Tesla watch", "12", "MuskBands", 9, true, "X AE
            A-12", true, "formal",
            funksjoner, true, 42, size, 0
        );
    }
}

```

Kode 6.1: Mock objekt for tester

6.4 testStepCounterInitialCount enhetstest

Første testen sjekker start verdien til skritteller. Denne skal da initialt være 0, da dette representerer antallet skritt en bruker starter på.

```

@Test
@DisplayName("Check initial steps counter is 0")
void testStepCounterInitialCount() {
    assertEquals(0, mockSmartWatch.stepCount);
}

```

Kode 6.2: Skritteller test 1

6.5 testStepCounterIncrements enhetstest

Denne testen kontrollerer første inkrement for funksjonen “stepCounter()” i klassen. Om klassen funksjon klarer å utføre ønsket inkrement, så vil denne testen fullføre med suksess.

```

public void stepCounter() {
    boolean stepDetected = detectStep();
    if (stepDetected) {
        stepCount++;
    }
}

private boolean detectStep() {
    // Simulerer steg for aa kunne utføre testing
    return true;

    // Her maa det byttes ut med faktiske funksjoner
    // som snakker med enheten via en port.
}

```

Kode 6.3: Funksjoner definert i smartWatch

Test funksjon:

```

@Test
@DisplayName("Step Counter increment check")
void testStepCounterIncrements() {
    mockSmartWatch.stepCounter();

    if (mockSmartWatch.stepCount == 1) {
        assertEquals(1, mockSmartWatch.stepCount);
    }
}

```

Kode 6.4: Skritteller test 2

6.6 testMultipleStepIncrements enhetstest

Denne testen er tilsvarende "testStepCounterIncrements", men denne tester for videre inkrementering av mock-objektets variabel "stepCounter".

```

@Test
void testMultipleStepIncrements() {
    for (int i = 0; i < 10; i++) {
        mockSmartWatch.stepCounter();
    }

    assertTrue(mockSmartWatch.stepCount > 8, "Step should
        increment.");
}

```

Kode 6.5: Skritteller test 3

6.7 Resultat av tre enhetstester

```

Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time
elapsed: 1.075 s -- in IOTSmartWatchTests

```

Results:

```

Tests run: 3, Failures: 0, Errors: 0, Skipped: 0

```

```

-----
BUILD SUCCESS
-----

```

```

Total time: 3.330 s
Finished at: 2024-10-24T12:22:04+02:00
-----

```

Kode 6.6: Resultat av tester

Modeller

7.1 Mockups

Som utgangspunkt for applikasjonens visuelle arkitektur er det laget en rekke hybride wireframes som dekker krav i sprint 1. Det er lagt til kontekstuekt innhold på flere overskrifter, knapper og paragrafer for å beskrive hensikten, men lite fokus på det estetiske. Enkelte komponenter er mer detaljerte for å fremheve navigasjon og vise ulike funksjonalitet som sluttbruker kan justere på IoT enheter.

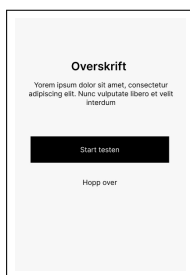
Opprinnelig skulle “mine produkter” være appens forside for innloggede brukere, men etter å ha diskutert første utkast er det nødvendig å ha en samleside for fremhevet informasjon. I tillegg til å være en app for styring og veiledning, ønsker bedriften å fremheve produktene de selger. Designing av hjemmeside blir et nytt krav i sprint 2.

Figma link til mockups finnes [her](#)

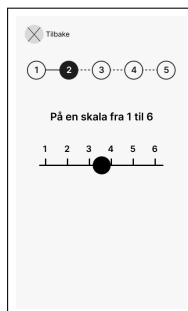
7.1.1 Kompetansetest

Ved registrering blir bruker anbefalt å ta en kompetanse test for å velge et oppsett som definerer applikasjonens utseende. Testens spørsmål er i form av en flervalgs løsning. Etter testen blir bruker henvist til et anbefalt oppsett, men får muligheten til å velge annet oppsett.

Tallene indikerer hvor mange steg testen har og hvilken side brukeren er på.



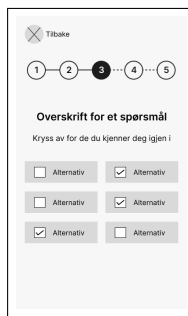
Figur 7.1: Kompetansetest - Start



Figur 7.2: Kompetansetest - Velg et tall



Figur 7.3: Kompetansetest - Ett valg



A mockup of a competency test question screen. At the top, there is a 'Tilbake' button with a back arrow icon. Below it is a progress indicator with five numbered circles (1-5), where circle 3 is filled, indicating the current question. The main heading is 'Overskrift for et spørsmål' followed by the instruction 'Kryss av for de du kjenner deg igjen i'. There are six checkboxes arranged in three rows of two, each labeled 'Alternativ'. The first, third, and fifth checkboxes are checked.

Figur 7.4: Kompetansetest - Flervalg



A mockup of a competency test result screen. The heading is 'Testen er fullført!'. Below it, a message states: 'Basert på ditt svar anbefaler vi enkel modus. Du kan endre dette senere'. There is a large black button labeled 'Velg oppsett og fullfør'. Below this is a section titled 'Velg annet oppsett' which contains two sub-sections: 'Vanlig oppsett' and 'Avansert'. Each sub-section has a paragraph of placeholder text and a button labeled 'Velg vanlig' or 'Velg avansert' respectively.

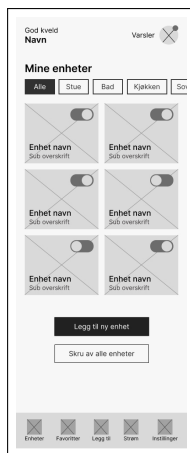
Figur 7.5: Kompetansetest - resultat

7.1.2 Konfigurerte enheter

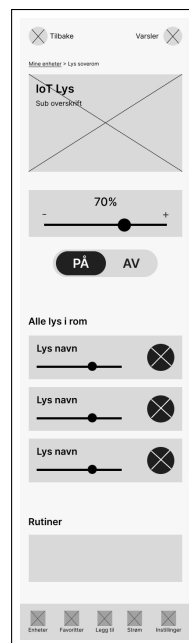
Mine enheter lister opp alle konfigurerte IoT enheter. Disse kan filtreres på ulike kategorier, som for eksempel rom i en bolig. Hver enkelt enhet har sin egen side, en mal med ulike komponenter som styrer oppførsel og rutiner. Enkle handlinger som å skru på/av en enhet kan gjøres direkte fra mine enheter siden uten å trykke inn på den.

For å vise at IoT enheter kan ha forskjellige UI komponenter for å styre oppførsel er det laget to wireframes, en for støvsugeren til personaen Sarah Johnsen og en for smartlys.

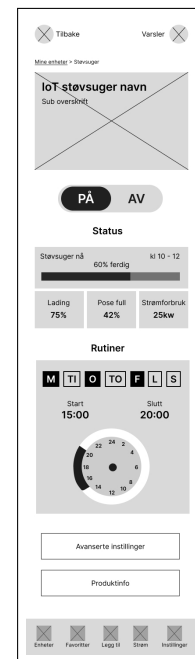
Støvsugeren kan vise status om lading, fyllingsgrad på posen og strømforbruk. Med lys kan alle skrues av/på eller dimmes, eller styres enkeltvis. Felles for enhetene er at de burde ha automatiserte rutiner som brukeren kan sette.



Figur 7.6: Mine enheter - Dashboard



Figur 7.7: Mine enheter - Lys



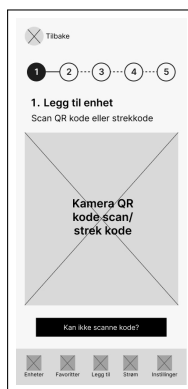
Figur 7.8: Mine enheter - Støvsuger

7.1.3 Installasjonsguide - enkel

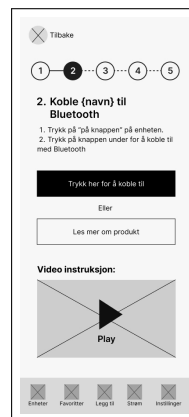
Steg for steg veiledning for installering av en enhet dersom bruker har valgt simpelt UI oppsett (burde kunne endres uten at det påvirker annen funksjonalitets utseende). Denne installasjonen konfigurerer kun det nødvendige for at enheten skal fungere for normalt bruk. Hver side gir bruker en tekstlig beskrivelse og en video eller bilde med informasjon om hvordan hver spesifikke del konfigureres.

Skissene under viser hvordan å finne enheten raskt ved å skanne en kode og koble til med bluetooth. Dersom QR eller strekkode ikke kan skannes vil bruker få alternative måter å finne produktet, som å søke etter produkt. Skisse to og tre viser ulike tilstander for en side for å koble til bluetooth. Det vil være elementer på siden som har subtile animasjoner som hjelper bruker å forstå når en enhet er koblet til telefonen.

Når konfigurering er gjennomført vil bruker få valget om å se den konfigurerte enheten eller legge til flere.



Figur 7.9: Installasjonsguide - SKann produkt



Figur 7.10: Installasjonsguide - Koble til telefon



Figur 7.11: Installasjonsguide - Kobling vellykket



Figur 7.12: Installasjonsguide - Ferdig

7.1.4 Installasjonsguide - Avansert

Avanserte brukere får presentert et annerledes oppsett for konfigurering av enheter med et fullstendig oppsett og flere alternativer. Dette oppsettet gir fullstendig oversikt på en side, hvor hvert steg befinner seg i en nummerert tab som strykes ut ved fullføring. Brukeren kan likevel gå bakover i prosessen.



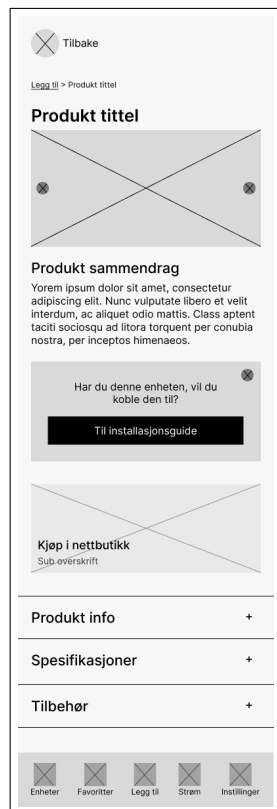
Figur 7.13: Installasjonsguide - Avansert

7.1.5 Produkt artikler, varslinger, annet

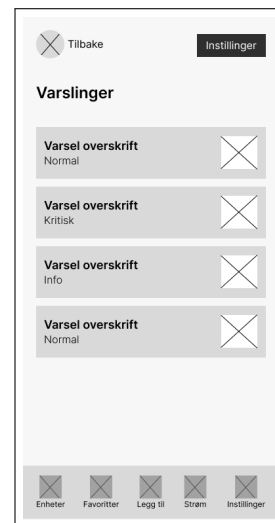
Produktartikler er egne mal-sider for markedsføring av bedriftens IoT enheter direkte i appen, og teknisk informasjon som er nyttig for forbruker. Artikler navigeres til på flere måter, gjennom appens forside (ikke skissert enda), mens en konfigurerer en enhet eller under siden for aktive enheter. Artikler inneholder også en liste over nødvendige eller funksjonelt avhengige enheter.

Sider som navigeres til på ulike måter vil ha relative brødsmulestier som indikerer hvor brukeren opprinnelig kom fra. Det er nødvendig hvis for eksempel en bruker går inn på en produkt artikkel under konfigurasjon av enhet.

Appen vil ha et eget varslingssenter som gir beskjed til brukeren når en enhet trenger oppmerksomhet. Bruker kan endre hyppigheten av varslinger under en egen side.



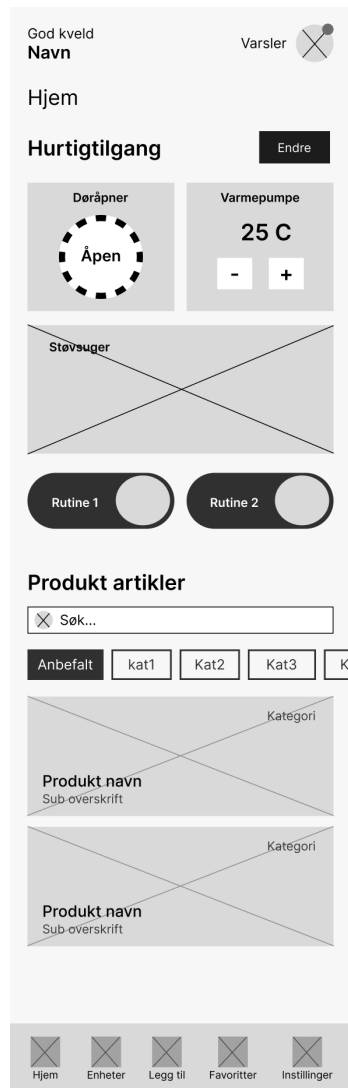
Figur 7.14: Produktartikkel



Figur 7.15: Varslignssenter

7.1.6 Forside kontrollpanel

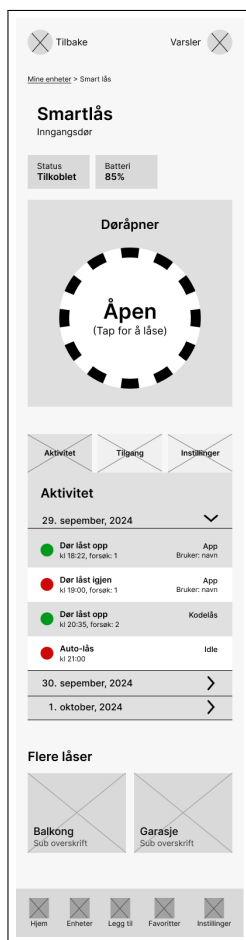
Forsiden fungerer som en samleside applikasjonen som kan justeres etter brukerens behov og interesse. I figur 5.1 ser en bare to synlige komponenter, men det kan potensielt være flere komponenter som kan skjules eller legges til. Den første komponenten er hurtigtilgang og makro styring av enheter, den neste er nyttig informasjon og markedsføring av produkter som bedriften selger.



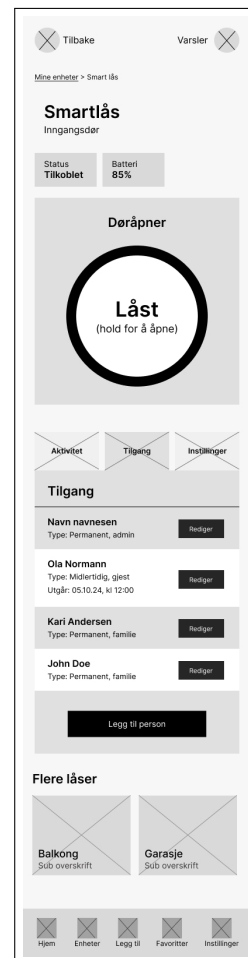
Figur 7.16: Hjem Dashboard

7.1.7 IOT enhet - smart lås

Figur 5.2 og figur 5.3 visualiserer IOTDoorLock klassen og et utkast av dens egenskaper og metoder. Figurene viser hvordan låsens ulike tilstander styres ved store knapper og enkle klikk. Denne klassen er tiltenkt å implementeres i selve prosjektet og kommer til å ha flere egenskaper for å dele ut tilganger og loggføre når personer låser opp eller igjen dører.

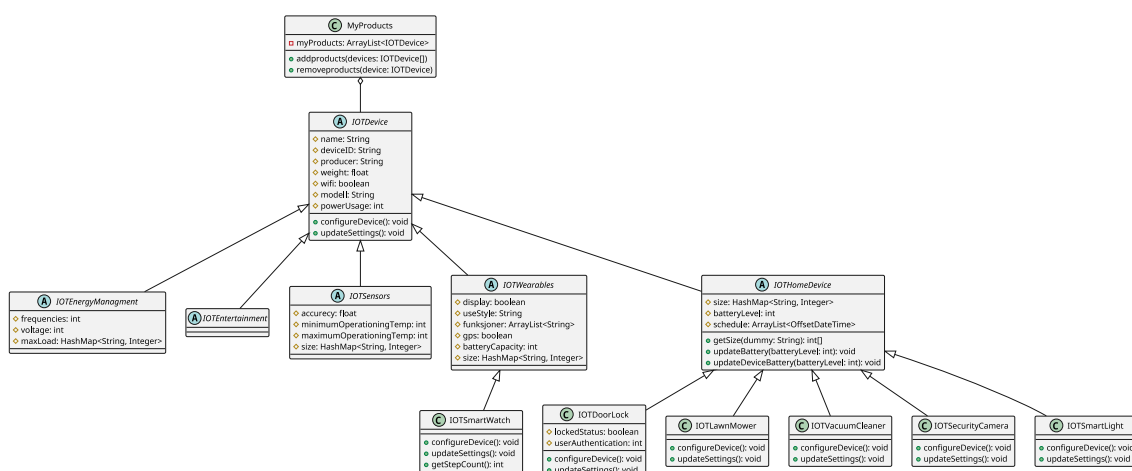


Figur 7.17: Smart lås - aktivitet



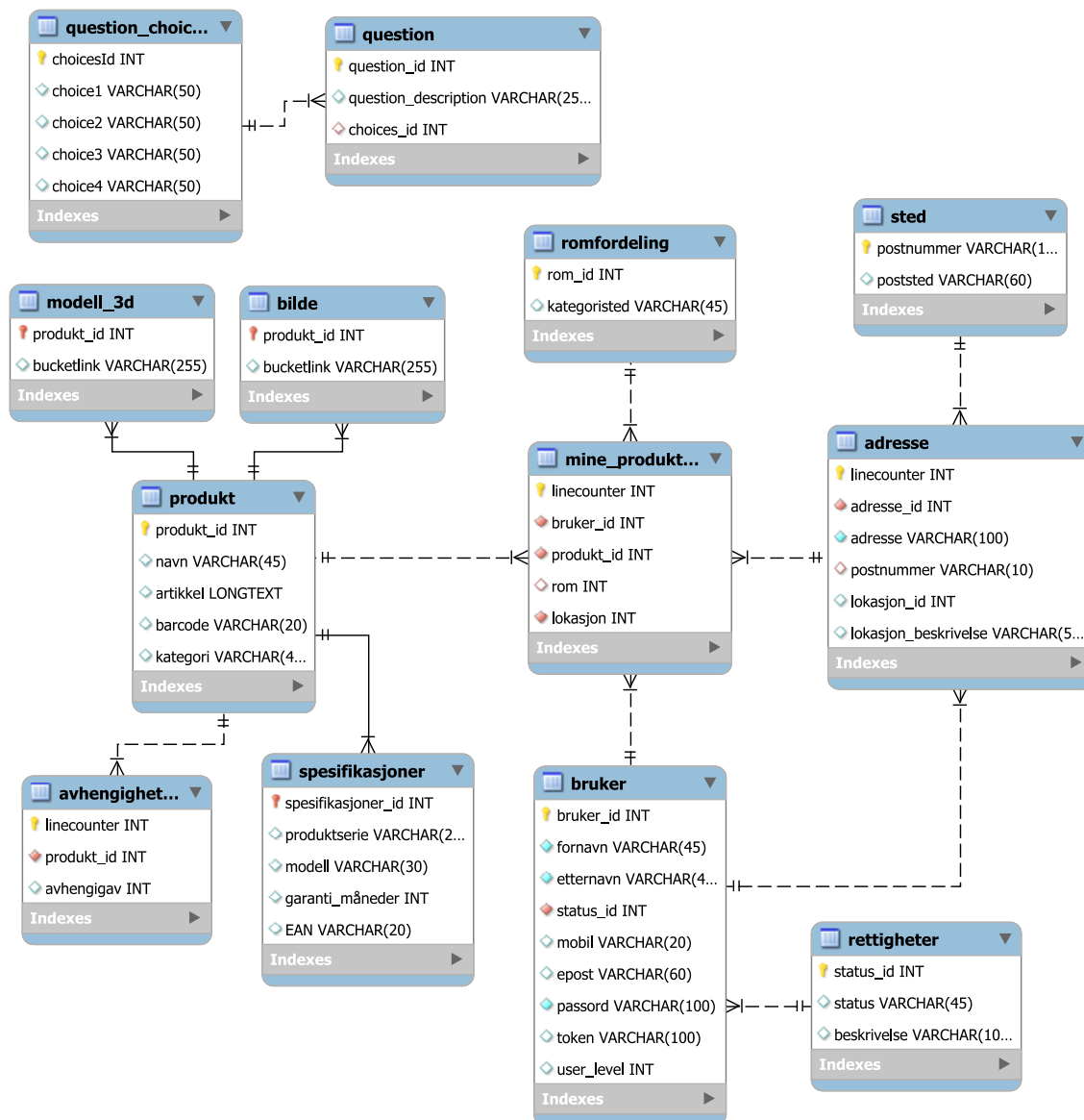
Figur 7.18: Smart lås - tilganger

7.2 Klassediagram



Figur 7.19: Klassediagram

7.3 EER-diagram

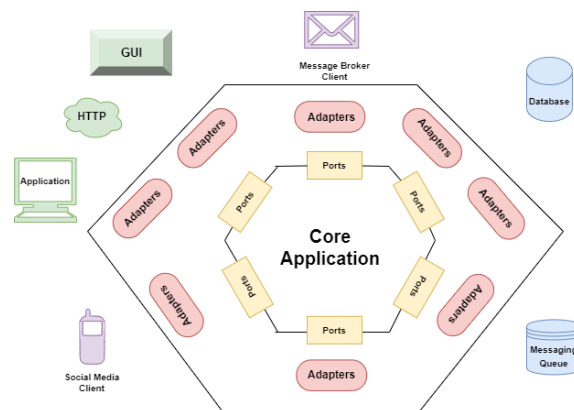


Figur 7.20: EER-diagram

7.4 Arkitektur

Applikasjonen skal utvikles med en heksagonal arkitektur som vil gjøre det gunstig å bytte ut teknologier og frontend, uten at det påvirker kjernefunksjonalitet. Til nå er prosjektet satt opp i en enkel modul med enkelte porter og adaptore som kommuniserer med databasen og GUI. Deler av prosjektet er sterkt tilkoblet, men vil gå over til mer løst koblet kode i sprint 3 da det blir mer fokus på arkitektur før utvikling av kjernekode fortsetter. I sprint 3 og frem mot prosjektlevering skal det utvikles mange

flere usecases, DTO's og refaktorere prosjektet i flere moduler. En overgang til full heksagonal arkitektur er påbegynt, men vil ikke ferdigstilles før i sprint 3.



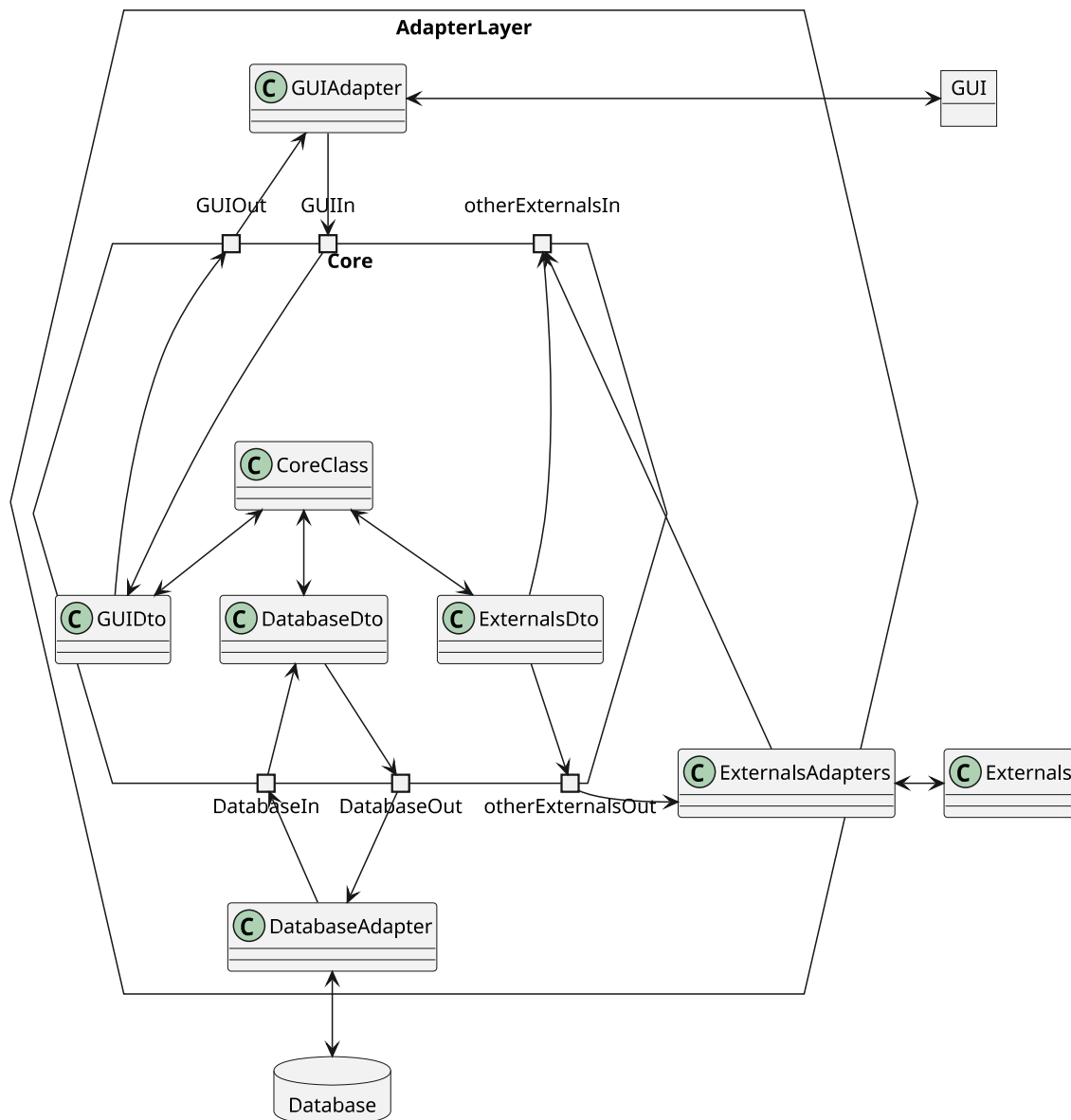
Figur 7.21: Heksagonal arkitektur[5]

Det er tiltenkt å dele inn prosjektet i modulene:

- Core - IOTEnhetsklassene
- User - Bruker klasser og klasser som tilhører bruker
- RepositoryTests - Test modul for å teste generell tilkobling til database
- FrontEnd - modul for alt frontend
- API - Inneholder dto's for usecasene og annen kode rundt api
- Security - krypterings algoritmen og annen kode som håndterer sikkerhet
- Utility - Denne blir bare laget vis vi blir å ha nok utility klasser til at det er relevant
- Configuration - Konfigurering av prosjektet samt kompetansetesten som skal sette vilken konfigurasjon brukeren skal ha

Swing som GUI rammeverk skal byttes ut i neste sprint, dette blir en reel test om fortsatt tilfredsstillende kravene til heksagonal arkitektur. Dersom det blir implementert riktig vil overgangen kun kreve å skrive et nytt adapter. Her er det også en case av fordelene med denne arkitekturen.

heksagonal arkitektur er nytt for de fleste medlemmene som medfører en del prøving og feiling, i tillegg litt usikkerheter. Her er det viktig med god kommunikasjon og strategi for å få hele prosjektet over på denne arkitekturen.



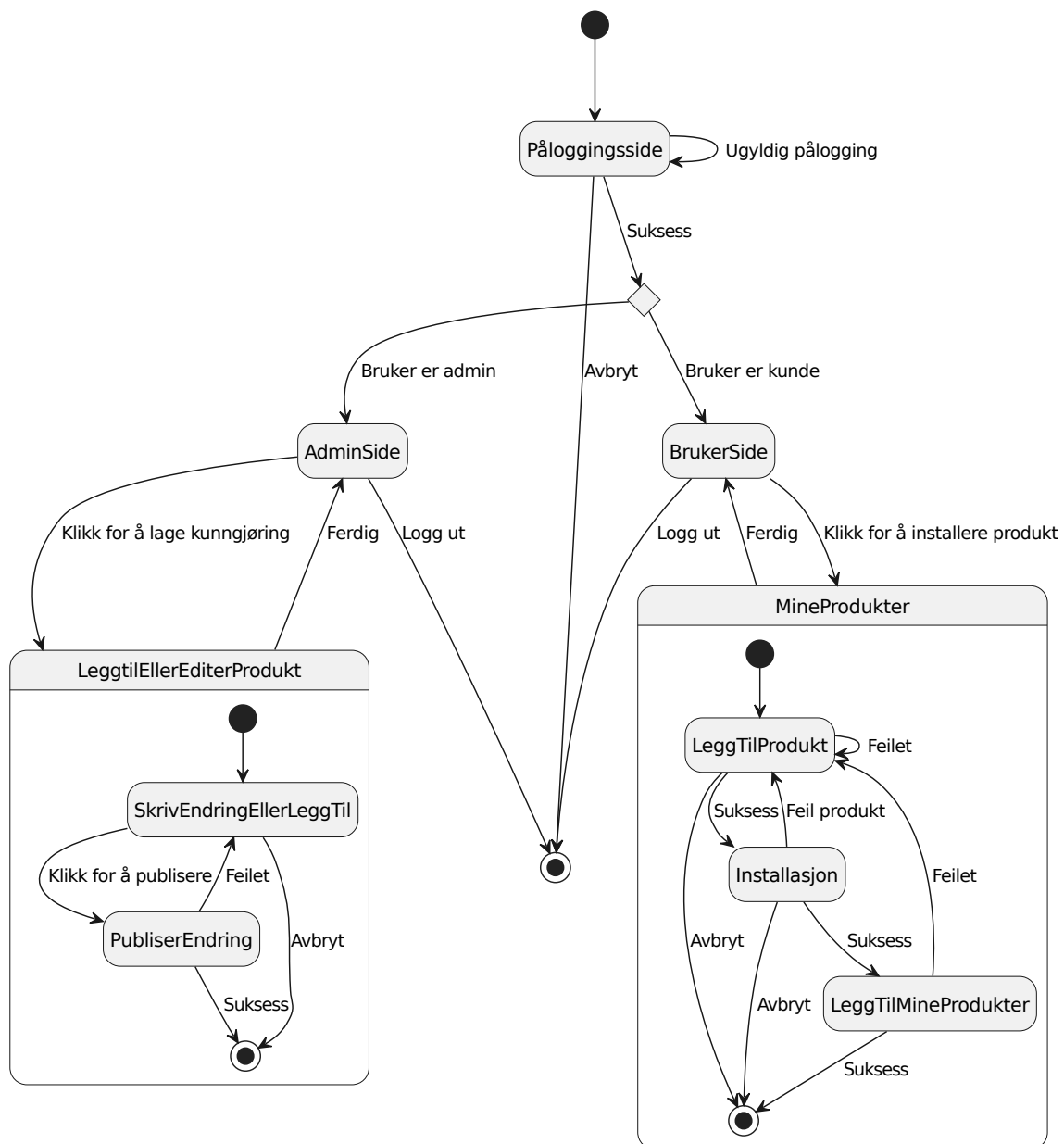
Figur 7.22: Heksagonal arkitektur - UML

7.5 Login flytskjema

Vedrørende innloggings funksjonalitet figur 7.23, så er dette en løsning som ville blitt satt under videre utvikling. Med dette så menes det at produktet per i dag ikke har alle løsninger som diagrammet henviser til.

Det er i dag satt opp til forskjellige tilgangsnivåer på brukere i databasesystemet, og dette vil videre kunne brukes til å implementere aksesserings nivåer for individuelle brukere. Dermed ar alle funksjoner som finnes utenfor funksjons boksene (LeggTilEllerEditorProdukt og MineProdukter) implementert. Ved videre utvikling av dette produktet ville det blitt jobbet med å implementere en løsning som

gjør at bruker kan legge til egne produkter i “Mine IOT produkter”, samt en funksjonalitet for ansatte å legge til og editere produkter uten å måtte håndtere dette direkte i databasen.



Figur 7.23: Login flytskjema

Drøfting

Under kodingen av applikasjonen, fant vi funksjoner som burde være med for en fullstendig applikasjon. Disse får vi ikke laget innen fristen, så å diskutere disse er viktig for å danne et bilde av fremtidige planer og implementasjoner, samt mulighetene og utfordringene disse bringer.

8.1 Krav til prototypen

8.1.1 Tillatelser

For å kunne lage de ønskede fremtidige funksjonene, kan vi måtte trenge å ha adgang til ting som enhetens kamera, potensielt mikrofon, WIFI, Bluetooth, osv. For en viktig del av å lage programvare er å lage bruker opplevelsen enkel, smidig og effektiv, og det kan derfor være nyttig å kunne bruke en enhets kamera for å for eksempel kunne skanne en AR-, QR- strekkode, istedenfor å måtte manuelt skrive inn ett serie nummer.

Samtidig kan det å etterspørre tillatelser til for eksempel en enhets kamera eller mikrofon være uheldig på personvern fronten og bruker opplevelsen, avhengig av hvordan dette implementeres.

8.1.1.1 Kamera

Ett av våre fremtidige mål med applikasjonen er å kunne bruke enhetens kamera til å skanne QR koden til smartenhetene, for å koble disse sømløst til. Det finnes flere løsninger for å implementere en slik funksjon og valget av hvilken må baseres på kravene til kunden og relevante sikkerhets og personverns reglementer, som for eksempel GDPR.

En måte å aksessere en enhets kamera, er å bruke en løsning der applikasjonen har permanent tilgang til en enhets kamera under bruk av applikasjonen. Dette er gunstig for bruker opplevelsen, da brukeren kun opplever å bli spurt 1 gang og ikke mer for å aksessere funksjonen. Dette er derimot negativt på personvern og sikkerhet siden, for om dette ikke blir håndtert på en riktig måte kan dette medføre brudd på GDPR og vi skaper muligheten for misbruk av kameraet av eksterne kilder.

En annen måte å oppnå tilgang til enhetens kamera, er med «tillatelsesprompter». Disse promptene dukker opp på brukerens skjerm for hver gang kamera funksjonen kreves. Ved å svare ja på promten gis det midlertidig tilgang til kameraet for applikasjonen. Denne løsningen kan negativt påvirke brukeropplevelsen, da hyppige «pop-ups» kan skape en ubehagelig opplevelse for brukeren. Personvern og sikkerhets messig er dette en god løsning, da det kun gis midlertidig adgang til en sensitiv ting.

Alt dette må tas i betraktning før noe som dette implementeres og dette gjelder flere av tillatelsene. Tilgang til Mikrofon, WIFI, Bluetooth kan vurderes og drøftes liknende, men det er likevel forskjeller mellom disse tillatelsene, med tanke på hvor «sensitive» de er.

8.1.1.2 Kalender

Vi har også hatt planer om integrering av planleggings funksjoner, og dette kan vi oppnå ved hjelp av tillatelse til å bruke enhetens kalender, gjennom et API. Dette kan i kontrast til kamera og mikrofon adgang sees på som en mindre farlig tillatelse for appen å ha, da kalender data ikke er like sensitivt som kamera og mikrofon data. Dette gjør det derfor mer relevant å bruke en løsning som er mer praktisk enn trygg.

8.1.2 Gjenkjenn Enhet

Ved hjelp av en kamera tillatelse løsning, skal vi kunne implementere en form for QR / bar kode leser noe som kan lette prossessen for tilkobling av smartenheter for brukeren og er et viktig neste steg for applikasjonen vår. Det finnes flere biblioteker for slikt, og et eksempel ZXing.

Om vi velger å bruke ZXing, kan fremgangsmåten se slik ut etter forslag fra [ChatGPT](#)

8.1.3 Varsel/ping funksjoner

Varsler er også noe vi ser på som en viktig ting å ha med i appen vår, for å ha en måte å informere brukeren på. Varsler er også ganske standardisert i dagens apper, og å ikke ha de med kan virke ukjent og upraktisk for brukeren. Appens varsler vil hovedsaklig være i form av, informering om passering av satte grenser for, for eksempel maksimalt strømforbruk, lite batteri på smartenhet, osv. Altså status på definerte grenser, enheter og potensielt app oppdateringer og diverse.

I praksis kan dette se slik ut, hvis en antar at brukeren allerede har sine enheter oppkoblet, programmet satt opp og gitt programmet tilgang til varslinger hos enheten:

1. Gressklipperen passerer grensen for «Lavt batteri»
2. Status til gressklipperen blir satt til «Lavt batteri»
3. Statusen «Lavt batteri» trigger en funksjon «send_varsel_til_bruker_om_lavt_batteri»
4. Brukeren får et varsel på enheten sin om dette.

8.1.4 Produkt scan

Etter å ha fått på plass scanne funksjonalitet, må vi lage kode for å hente ut produktet som tilhører denne bar koden i databasen. Her er et forslag til hvordan hvert steg i prossessen kan se ut og hva koden kommer til å måtte omhandle:

1. Bruk scanne resultatene til å trekke ut produkt data fra databasen
 - (a) Kobling til databasen
 - (b) Videre sending av scanne resultat til SQL spørring

- (c) Hente ut data
- 2. Sjekke om produktet allerede er lagt inn i appen
 - (a) Om dette er tilfelle, spør bruker om en ønsker å legge til et til produkt eller overskrive det gamle eller droppe det
 - (b) Om ikke, ha funksjon som legger produktet inn og sender brukeren til oppsett siden av en ny enhet.

# produkt	navn	artikkel	barcode	kategori	spesifikas	produktse	modell	garanti_m	EAN
'1'	'Samsung	'En smart l	'S1234567	'Electronic	'1'	'Samsung	'SmartBulb	'24'	'S1234567890123'
'2'	'Philips Sm	'Smart lysp	'P9876543	'Electronic	'2'	'Philips Hu	'SmartBulb	'36'	'P9876543210123'
'3'	'Nokia Sm	'Lyspære r	'N4567890	'Electronic	'3'	'Nokia Hor	'SmartBulb	'12'	'N4567890123456'
'4'	'Apple Sm	'Lyspære s	'A3216543	'Electronic	'4'	'Apple Ligh	'SmartBulb	'24'	'A3216549870987'
'5'	'Samsung	'Overvåkin	'S2345678	'Security'	'5'	'Samsung	'Cam 2000	'12'	'S2345678901234'
'6'	'Philips Sm	'Sikkerhet	'P8765432	'Security'	'6'	'Philips Wa	'SmartCar	'24'	'P8765432109876'
'7'	'Nokia Sm	'Intelligent	'N1357924	'Security'	'7'	'Nokia Visi	'SmartCar	'12'	'N1357924680123'
'8'	'Apple Sm	'Kamera m	'A2468135	'Security'	'8'	'Apple Sec	'SmartCar	'24'	'A2468135790987'
'9'	'Samsung	'Roboterkl	'S9876543	'Gardening'	'9'	'Samsung	'Mower Pr	'36'	'S9876543210123'
'10'	'Philips Sm	'Kraftig rob	'P1234567	'Gardening'	'10'	'Philips Ec	'Mower X'	'24'	'P1234567890456'
'11'	'Nokia Sm	'Effektiv gr	'N5432167	'Gardening'	'11'	'Nokia Gre	'Mower Y'	'12'	'N5432167899876'
'12'	'Apple Sm	'Intelligent	'A1597534	'Gardening'	'12'	'Apple Mov	'Mower Z'	'36'	'A1597534680123'
'13'	'Samsung	'Robotstøv	'S1112223	'Sanitation'	'13'	'Samsung	'Vacuum F	'12'	'S1112223334445'
'14'	'Philips Sm	'Støvsuger	'P4445556	'Sanitation'	'14'	'Philips Pu	'Vacuum X'	'24'	'P4445556667890'
'15'	'Nokia Sm	'Effektiv st	'N7778889	'Sanitation'	'15'	'Nokia Sm	'Vacuum Y'	'12'	'N7778889991112'
'16'	'Apple Sm	'Intelligent	'A0001112	'Sanitation'	'16'	'Apple Cle	'Vacuum Z'	'36'	'A0001112223334'
'17'	'Samsung	'Smartklok	'S1237894	'Wearable'	'17'	'Samsung	'Watch A'	'24'	'S1237894560123'
'18'	'Philips Sm	'Klokke me	'P9871236	'Wearable'	'18'	'Philips Fit	'Watch B'	'36'	'P9871236544567'
'19'	'Nokia Sm	'Smartklok	'N6543215	'Wearable'	'19'	'Nokia Tra	'Watch C'	'12'	'N6543219876543'
'20'	'Apple Sm	'Premium	'A3219876	'Wearable'	'20'	'Apple Ser	'Watch D'	'36'	'A3219876540987'

Figur 8.1: Produktliste

8.1.5 Justering av varsler

Mulig løsning for push notifikasjoner.

En av de flere oppgaver som ikke ble løst under dette prosjektet, men som ville ha blitt implementert ved videre utvikling er push notifikasjon til brukeren.

Grunnlaget for at dette ville blitt en prioritert oppgave ved videre utvikling, er at dette ville kunne dekke for forskjellige typer behov.

Fra brukers perspektiv ville dette vært effektivt i forhold til å få tilbakemeldinger på IoT enhets oppgaver og status rapportering. Slik som notifikasjon om lavt batteri eller tilbakemelding på utførte og planlagte oppgaver. Det vil også være et sikkerhets aspekt her som kunne notifisert bruker om noen har logget inn på brukers konto.

Fra produsentens perspektiv vil dette være et svært effektivt verktøy for å kunne kommunisere diverse informasjon til sine kunder. Slik som kampanjer, produkt oppdateringer, sikkerhets varsler og mer.

Til tross for at denne funksjonen ikke er implementert i produktet, så er dette en mulig løsning som ville blitt vurdert å implementere om produktets utvikling hadde fortsatt. Dette kan leses på Google sin firebase dokumentasjon [her](#)

8.1.6 Applikasjonens systemsikkerhet

Når vi går inn på systemsikkerhetsdelen for appen vår, så er det veldig viktig for oss at personvern, persondata og generelt all sensitiv informasjon om kunden vår ikke blir lekket eller hacket. Ikke bare sensitiv informasjon om kunden, men om produktene til kundene også.

Med tanke på at alt er koblet opp sammen på ett sted, så vil et brudd være veldig kritisk. Informasjon og tilgang til wifi, hjemmeadresser, låser, alarmsystemer og annen kritisk informasjon for anonymitet og privatliv kan kunne komme på avveie som vil føre til konsekvenser både for oss og kundene. Det er derfor avgjørende at et godt sikkerhetssystem kommer på plass tidlig, spesielt om virksomheten vokser og tiltrekker seg oppmerksomhet.

Implementering av sikkerhetstiltak på en IOT-applikasjon er utfordrende for oss, nettopp på grunn av hvor kompleks systemet kan være samtidig med balansen mellom brukervennlighet og sikkerhet.

I tillegg til datasikkerheten og systemsikkerheten, er det like viktig for oss at IOT-applikasjonen vår holder personvern lover og regler (GDPR), for å sikre at kundenes personopplysninger behandles på en lovlig og trygg måte. GDPR krever at data behandles med hensyn til konfidensialitet og at kundene får beskjed om hvordan dataen deres samles og brukes (cookies).

Systemsikkerheten er veldig viktig for oss, derfor ligger den høyt prioritert på agendaen for neste iterasjon av applikasjonen fremover. Dette for å både ha kundenes tillit og slippe store bøter som ville vært ganske avgjørende for virksomheten.

8.1.6.1 Hashing av passord

I applikasjonen så har det blitt brukt et eksternt verktøy for å kunne kryptere og dekryptere passord. Dette blir brukt før lagring av passord, og det blir brukt ved autentisering. Denne avhengigheten ligger i pom.xml og blir kalt JBCrypt fra mindrot. Dette er med på å øke sikkerheten i tilfelle databasen skulle vise seg å bli hacket.

8.1.6.2 Cookies

Ved innlogging har det også blitt lagt til en cookies funksjonalitet. Hver gang en bruker logger inn blir det lagret en cookie lokalt med en token som har blitt generert. Denne token blir også lagret for hver bruker i databasen. Dermed er det lettere å beskytte visse endepunkt ved å sjekke at verdien på token lokalt samstemmer med verdien på token i databasen. Dette er med på å øke sikkerheten til applikasjonen.

8.1.7 Målsetting for strømforbruk

En annen planlagt funksjon er å kunne informere og hjelpe brukeren med strømsparing ved hjelp av en bruker definert strøm forbruks grense. Denne grensen skal brukeren ha mulighet til å justere. Denne grensen er mer ment som et mål enn en grense, som skal virke som et hjelpemiddel for å oppnå redusert strømforbruk. Den skal fungere slik at når brukerens definerte strømgrense er blitt passert, skal brukeren informeres om dette og eventuelt få råd og informasjon om hvordan den totale strømbruken kan reduseres hos smartenhetene oppkoblet til appen.

I praksis kan dette se slik ut, hvis en antar at brukeren allerede har sine enheter oppkoblet, programmet satt opp og gitt programmet tilgang til varslinger hos enheten:

1. Bruker definerer en grense i appen, kanskje i form av en slider
2. Variabelen «strøm_maal_grense» blir satt til dette
3. Ved passering av den definerte verdien i «strøm_maal_grense» vil en funksjon «strøm_maal_passert» kjøre
4. «strøm_maal_passert» sender varsel til brukeren om at strømforbruken er passert det maksimalt ønskede forbruket.
5. Bruker mottar anbefalinger ved interaksjon med varslingen angående løsninger til problemet.

8.1.8 Monitorering av strømforbruk

Siden produktet vårt både skal holde kunder fornøyd, spare tid og være så bærekraftig som mulig, er monitorering av strømforbruk et viktig aspekt for å holde kontroll på energieffektivitet og forbruk. Magien bak denne monitoreringen er datainnsamling fra de ulike IOT-enhetene vi har som støvsugere, gressklippere, sikkerhetskamera etc.

Med datainnhenting for disse produktene kan kundene våres enkelt holde styr på hva som bruker mest strøm og hva som bruker minst. Dette gjør det enkelt for brukerne å velge hvilke enheter de vil justere bruken på for å redusere eventuelle ekstra kostnader.

Men hvordan gjøres dette? Enhetene har innebygde funksjoner og sensorer for å nettopp måle disse forbrukene. Dataene som samles sendes til servere som deretter blir visualisert i appen. Her får kundene oversikt og kontroll i sanntid. Utfordring med dette kan være nøyaktighet av data.

Denne implementasjonen av bærekraftig monitorering av strømforbruk kan også være en konkurransefordel for oss. Det er åpenbart i dagens samfunn stor etterspørsel etter miljøvennlige løsninger som tar hensyn til planeten så mye som mulig. Utviklingskostnadene kan i denne sammenhengen bli et krevende hinder for oss, sammen med begrensninger for teknologi og balansen mellom det og brukervennlighet. Dette gir oss potensiale for planer om fokusområder for videre utvikling.

Med dette skaper vi gode løsninger både for miljøet, kundene våres og oss selv.

8.1.9 Status monitorering

Program som lytter etter statuskoder fra kundens aktive IOT enheter. Dette kan være å ta imot status kode 0 som sier at alt går som normalt, eller kode 1 hvor støvsugeren er tom for strøm og kommer seg ikke tilbake til docken, da skal programmet sende ett push varsel til bruker basert på innstillingene valgt.

For å kunne gi brukeren beskjed når de må foreta seg noe i henhold til IoT-enheten, er det å kunne motta status fra IoT-enheten en vesentlig funksjon. Dette kan gjøres gjennom statuskoder som enheten sender ut til applikasjonen. For eksempel kan enheten sende statuskode 0 når alt fungerer som det skal, og 1 når enheten ikke kommer seg tilbake til ladedokken. For at dette ikke skal bli for innviklet for brukeren, vil applikasjonen gi en mer informativ melding til brukeren enn å bare sende en statuskode.

I denne MVP-en kommuniserer applikasjonen med IoT-enheten med statuskoder i tillegg til kodeord. Det er også lagt inn at batteriet til enheten tømmes underveis i kjøringen av programmet. Den gir beskjed til serveren når batteriet når 15% og 5%. Det å kunne vise disse meldingene i frontenden er neste steg i utviklingen av statusmonitorering, Det vil så være naturlig å prøve å få sendt status meldingene som push-varslar på mobile enheter.

Videre arbeid rundt statusmonitorering vil kreve en del undersøkelse om hvilke feil som eventuelt kan oppstå på en IoT-enhet, og hvordan man kan gi brukeren best mulig informasjon uten å overfylle varslingsloggen. Push-varselet burde også kunne endres i innstillingene slik at vi møter forskjellige brukeres behov.

8.1.10 Enhetsstyring

Det å kunne styre IOTEnheter er en vesentlig del av vår løsning. Gruppen har ikke muligheten til å programere opp mot en faktisk IOTEnhet, derfor har vi testet integrasjon mot en eventuel IOTEnhet gjennom å simulere en kobling gjennom portene på datamaskinen opp mot en falsk IOTEnhet. Den falske IOTEnheten er en enkelklasse som består for det meste av enkle variabler. Den er koblet opp mot adapter laget i vårt program, og kommuniserer med kjernen av programmet gjennom en server. Til nå har fokuset vært å få kommunikasjontil å gå smertefritt, det er derfor ikke blitt tid til å implementere dette i frontend. i tillegg så er det ikke en generell tilkobling som passer til alle mulige typer IOTEnheter.

Det er valgt ut en spesifikk type IOTEnhet. Dette er fordi poenget har vært å vise at konseptet fungerer og at vi kan få konseptet til. Ved videre utvikling vil det første steget være å gjøre kommunikasjonen så generell at det kan brukes mot flere typer IOTEnheter. Den vanligste måten å styre IOTEnheter på er gjennom IOT-APIer, det å programere opp mot API kan hjelpe oss med å få løsningen mer generell.

Det å koble løsningen sammen med databasen og frontend vil også gjøre brukeropplevelsen mye forbedret. MVP-en vår er delt i to deler hvor enhetsstyring blir en egen del skilt fra frontend delen.

Programmeringen av denne løsningen har vært krevende da det har dukket opp mange uventede feil underveis. Noe har vært i vår kontroll mens andre feil har vært utenfor vår kontroll. Kunnskapsnivået

på datanettverks programmering er ikke så høyt i gruppen og dermed har mye av tiden gått på å finne ut hvordan dette fungerer i forskjellige situasjoner. De største utfordringene har vært porter som blir overtatt av andre programmer, Programmering med mange tråder og serialiserings problemer. MVPen på enhetstyring fungerer basert på terminal commandoer og krever at en starter to til flere prosesser, alt etter hvor mange klienter man ønsker å teste kommunikasjonen med. Før innlevering har det vært en vellykket testet med 5 klienter tilkoblet på en gang.

8.1.11 Enhets-API

IOT enheter bruker API for å kommunisere med applikasjoner og dette gjør at programøren ikke nødvendigvis trenger å kunne dyptgående kunnskap om hvordan IOT enheten er satt sammen for å kunne programere opp mot den. MVP har ikke tatt i bruk IOT API er fordi det krever å ha en IOT enhet å teste mot og mye tettere samarbeid med IOT leverandørene, spesielt vis det er snakk om IOT API som ikke offentlig tilgjengelig såkalte Open API. Ved videre utvikling ville det vært naturlig å programere opp mot IOT API-er.

8.1.12 Installasjonsprotokoll

For å kunne implementere en installasjons protokoll i vårt produkt, er det nødvendig med en omfattende undersøkelse av ulike produkt leverandører og deres spesifikke krav og teknologiske løsninger. Hensikten med dette arbeidet er å oppnå en dypere forståelse av hvordan installasjon fungerer på tvers av forskjellige enheter, inkludert eventuelle tekniske begrensninger, krav til kompatibilitet, og standardiserte praksiser som allerede er etablert i bransjen.

Dette vil kreve en systematisk kartlegging av installasjonsprosesser, inkludert hvordan programvare distribueres, hvordan enheter konfigureres, og hvilke sikkerhetstiltak som må være på plass for å sikre en pålitelig og sikker installasjon. Det er sannsynlig at ulike leverandører har forskjellige fremgangsmåter og tekniske løsninger, noe som gjør det nødvendig å utvikle en fleksibel og skalerbar protokoll som kan tilpasses til ulike kontekster.

En slik tilnærming vil ikke bare bidra til å sikre en vellykket implementering av protokollen, men også posisjonere vårt produkt som en pålitelig og kompatibel løsning på tvers av en rekke enheter og plattformer. For å oppnå dette anbefales det å sette av ressurser til en grundig markedsanalyse, samarbeid med leverandører, og testing i kontrollerte miljøer før fullskala utrulling.

8.1.13 Avansert/Enkel brukergrensesnitt for applikasjonen

Vi hadde sett for oss en toggle-knapp funksjon for et bytte mellom et enkelt og et avansert brukergrensesnitt. Dette hadde vært en fin nøkkelfunksjon for å tilpasse applikasjonene for flere ulike brukergrupper. For eksempel kunne det for brukere med mindre teknisk erfaring, vært et enkelt grensesnitt som viser grunnleggende funksjoner som enkel styring og oppsett av enheter, oversikt over status. Mens for et mer avansert grensesnitt for brukere med mer teknisk erfaring, kunne applikasjonen hatt flere detaljer, konfigurasjons muligheter og teknisk analyse/data tilgjengelig.

Med en slik løsning har vi da fordelen med å nå en bredere kundegruppe, samt øke brukervennlighet. Det gir de erfarne mer kontroll mens de mindre erfarne ikke blir overveldet med informasjon og vanskelige navigeringer.

Utfordringen for oss med en slik implementasjon, ligger i design og implementasjon. Det skal helst være en enkel og 'smooth' overgang mellom de to modusene. Det er en vanskelig og tidkrevende funksjonalitet. Vi må ha balanse mellom enkelhet og god funksjonalitet uten at det blir for enkelt eller for avansert. Samtidig bør det heller ikke gå utover applikasjonens ytelse.

Denne løsningen kan i utgangspunktet øke brukeropplevelsen samtidig som det styrker applikasjonens popularitet for en bred mengde med kunder. For at alt dette skal fungere, krever det masse tid med testing av begge grensesnittene, både designmessig og funksjonelt for å sikre at så mye som mulig fungerer optimalt. Denne funksjonaliteten vil uten tvil gi god verdig for tid og eventuelt penger investert.

8.1.14 Mine produkter - Meny

Gruppen hadde sett for seg en praktisk og brukervennlig funksjon der kundene våres får en enkel oversikt over alle IoT-enhetene de eier og som de har koblet opp mot applikasjonen. Denne menyen gir en fin mulighet for å være et "sentralt området" type punkt i applikasjonen, som gjør det raskt å både administrere og finne de forskjellige enhetene.

Denne menyen vil gi kundene våres fordelen av å hode en godt oversikt over enhetene sine, spesielt om de har flere enheter i sin husholdning. De vil også kunne se status på hver eneste enhet de eier. Status på om for eksempel enheten er på eller ikke, om de vil skru på eller av lading og forandre innstillinger. Dette gir en god brukervennlighet så kundene slipper å navigere flere menyer for hver eneste enhet, men heller ha alt på et sted.

Utfordringer med denne menyen for oss blir godt design og ryddighet. Skal alle enhetene vises hele tiden? Skal man kategorisere dem etter hvilket rom i huset de er i? En blanding av alle? Her er det mye spennende ideer man kan komme med. Riktig status for enhetene er også veldig viktig, slik at kunden ikke får feilinformasjon plutselig.

Dessverre er dette en funksjon som må bli med i en senere iterering av applikasjonen, men ikke tvil om at den vil gjøre den veldig attraktiv.

8.1.15 Design og universell utforming

Videre utseende messig arbeid på brukergrensesnittet, vil inkludere arbeid på forbedring av eksisterende og fremtidig utseende for å oppfylle WCAG kravene for universell utforming. Dette er viktigere fremfor estetikk og elegans, spesielt om applikasjonen i dette stadiet fortsatt sees på som en MVP, der funksjon kommer før design.

WCAG kravene sier noe om utformingen av grensesnitt, slik at alle mennesker uansett funksjonsevne kan interagere med det.

Universell utforming betyr at brukarane – uavhengig av sine føresetnader – på ein god og enkel måte kan ta i bruk både nettsider, appar og automatar som dei møter i kvardagen. Dette er pålagt gjennom forskrift om universell utforming av ikt-løysingar. [6]

Listen av alle kravene kan en finne [her](#).

Etter endringene vil det også være nødvendig å gjennomføre tester med brukere, for å videre forbedre brukeropplevelsen og grensesnittet generelt. [7]

1. Sikre at dimensjonene av knapper, tekst og andre elementer er riktig og tilpasset slik at det er mulig å interagere med uten problemer
2. Sikre tilstrekkelig kontrast mellom tekst og bakgrunn, slik at innholdet er lesbart.
3. Gi brukeren en mulighet for å navigere seg og vite hvor de befinner seg i grensesnittet.
4. Gi brukeren mulighet for å endre knapper og tekst størrelse i innstillinger.
5. Tilby alternative løsninger og alternativ grensesnitt interaksjon for personer med funksjonsnedsettelse.

Etter endringene vil det også være nødvendig å gjennomføre tester med brukere, for å videre forbedre brukeropplevelsen og grensesnittet generelt.

8.1.16 Implementasjon av brukergrensesnitt

I utviklingen av vårt system har implementeringen av et grafisk brukergrensesnitt (GUI) vært en sentral del av arbeidet for å sikre en intuitiv og funksjonell løsning for sluttbrukerne. Vi startet med å implementere GUI ved hjelp av Java Swing. Valget av Swing som første verktøy ble gjort fordi det tilbyr en forenklet måte å bygge et GUI som kommuniserer direkte med kjernestrukturen i Java-koden vår. Denne tilnærmingen gjorde det mulig å raskt utvikle en grunnleggende versjon av grensesnittet, samtidig som vi sikret en tett integrasjon med den underliggende logikken.

Etter den innledende utviklingsfasen vurderte vi web-baserte alternativer og forsøkte å implementere Vue.js som en del av GUI-løsningen. Tanken bak dette skiftet var å dra nytte av den større fleksibiliteten og skalerbarheten som et moderne frontend-rammeverk som Vue kunne tilby. Ved å dra nytte av nettlesere vill det tillate å vise grensesnittet på tvers av skjermstørrelser og enheter, men som samtidig bevarer kommunikasjon med kjernestrukturen i Java-koden. Dessverre møtte vi utfordringer med å få Vue.js til å fungere i praksis, noe som resulterte i at vi ikke kunne integrere det på en tilfredsstillende måte i systemet vårt.

Som konsekvens besluttet vi å endre tilnærming og implementere grensesnittet ved hjelp av HTML, CSS og Javascript, uten hjelp av eksterne Javascript rammeverk som Vue.js.

Til tross for dette gjenstår det enkelte deler av systemet som ikke har en GUI-løsning i dag. Disse funksjonalitetene er imidlertid fullt operative gjennom terminalen. Et eksempel er kommunikasjonen mellom mock-up IOT-enheter og kjernestrukturen i Java-koden, som fungerer problemfritt via terminalbaserte kommandoer. Dette illustrerer behovet for videreutvikling av GUI for å dekke hele produktets funksjonalitet og dermed sikre en helhetlig brukeropplevelse.

Denne progresjonen reflekterer utfordringene og beslutningene vi har tatt i arbeidet med å balansere funksjonalitet, brukervennlighet og teknologiske muligheter i utviklingen av systemets GUI.

8.1.17 Kompetansetest

Når en ny bruker logger inn i systemet for første gang, gjennomføres en initieringsprosess som tar sikte på å kartlegge brukerens kompetansenivå. Dette rakk vi ikke å implementere for denne gang, men basert på personas og scenarioer er det logisk å implementere et spørreskjema for nye brukere senere i utviklingen. Resultatet av dette skjemaet tilpasser opplevelsen ut fra brukerens tekniske kompetanse om bruk av IoT-enheter.

Denne prosessen begynner med en [kompetansetest](#) som består av seks spørsmål. Spørsmålene er designet for å evaluere hvor kjent eller trygg brukeren er med å bruke IoT-enheter, og brukeren kan svare ved å velge ett av fire alternativer. Disse spørsmålene dekker de mest essensielle kompetansene, og svarene som gis analyseres i sanntid i backend.

Et forslag til hvordan spørsmålene kan se ut:

8.1.17.1 Forslag til spørsmål

Spørsmål 1:

Hvor komfortabel er du med å bruke smarttelefonen til å kontrollere enheter i hjemmet ditt?

1. Ikke komfortabel
2. Litt komfortabel
3. Komfortabel
4. Svært komfortabel

Spørsmål 2:

Har du satt opp IoT-enheter hjemme tidligere?

1. Ingen erfaring
2. Noe erfaring
3. Mye erfaring
4. Svært mye erfaring

Spørsmål 3:

Hvor komfortabel er du med å feilsøke problemer med smarte enheter?

1. Ikke komfortabel
2. Litt komfortabel
3. Komfortabel
4. Svært komfortabel

Spørsmål 4:

Hvor interessert er du i å automatisere hverdagsoppgaver ved hjelp av smarte enheter?

1. Ikke interessert
2. Litt interessert
3. Interessert
4. Svært interessert

Spørsmål 5:

Hvor trygg føler du deg på å sette opp enheter som krever nettverksforbindelse?

1. Ikke trygg
2. Litt trygg
3. Trygg
4. Svært trygg

Spørsmål 6:

Hvor viktig er sikkerheten til dine IoT-enheter for deg?

1. Ikke viktig
2. Litt viktig
3. viktig
4. Svært viktig

Spørsmål generert med ChatGPT, link til samtale [her](#)

8.1.18 Analyse av kompetansenivå

Basert på brukerens svar genereres en poengsum som brukes til å tilordne brukeren ett av tre definerte kompetansenivåer: nybegynner, regelmessig bruker eller avansert bruker. Dette nivået bestemmes gjennom en algoritme som vurderer brukerens poengsum opp mot verdier definert i systemet. Den mest realistiske situasjonen er at avanserte brukere, som personasen Priya, får tilgang til detaljerte finjusteringer av enhetene samt detaljert statistikk og feilsøking. Mindre kyndige brukere får et enklere oppsett med færre detaljstyringer.

Etter at et nivå er fastsatt, vil systemet foreslå det estimerte brukerinnstillingsnivået til brukeren. Brukeren presenteres deretter for en dialog hvor de kan velge å akseptere det foreslåtte nivået eller manuelt justere sitt brukerinnstillingsnivå basert på egne preferanser. Hvert svar benyttes i en utregning som til slutt anbefaler et oppsett for brukeren, som fritt kan velge noe annet enn det som blir presentert. Oppsettet brukeren velger lagres som en ID i databasen og kan senere justeres under brukerens profilinnstillinger. Dette gir brukeren en balanse mellom systemstyrt veiledning og personlig autonomi.

Denne tilnærmingen gir en optimal tilpasning av systemets funksjonalitet til brukernes ferdigheter, samtidig som den legger til rette for brukerens individuelle behov. Ved å implementere en slik prosess reduseres risikoen for overbelastning for nybegynnere, mens avanserte brukere raskt kan få tilgang til mer komplekse funksjoner. Systemets fleksibilitet bidrar dermed til en positiv brukeropplevelse og en mer effektiv utnyttelse av systemets ressurser.

8.2 Bruk av rammeverk

I dette prosjektet har det blitt brukt rammeverk for å forenkle arbeidet som har blitt gjort. For å kunne lage en backend i Java ble det brukt Javalin, mens for å kunne kommunisere med Azure databasen ble det brukt en MySQL JDBC driver.

Prosjektet i seg selv har blitt holdt veldig enkelt. Og bruken av rammeverk har blitt holdt til et minimum for å kunne lære mer koding og bruke det i prosjektet. Ta f.eks backend - det kunne ha blitt brukt Spring Boot rammeverket som hadde gjort jobben veldig mye enklere. Det er mange andre avhengigheter som kunne ha blitt brukt, men da ville ikke læringen vært den samme. Å gjøre ting litt vanskeligere kan i noen tilfeller også være mer lærerikt.

For frontend ble det først brukt Vue slik som ble vist i Programmering 2, men av en eller annen grunn oppstod det feilmeldinger ved bruk av WebJars, og da ble det bestemt å gå bort ifra bruken av frontend rammeverk.

For testing av kode ble det brukt JUnit.

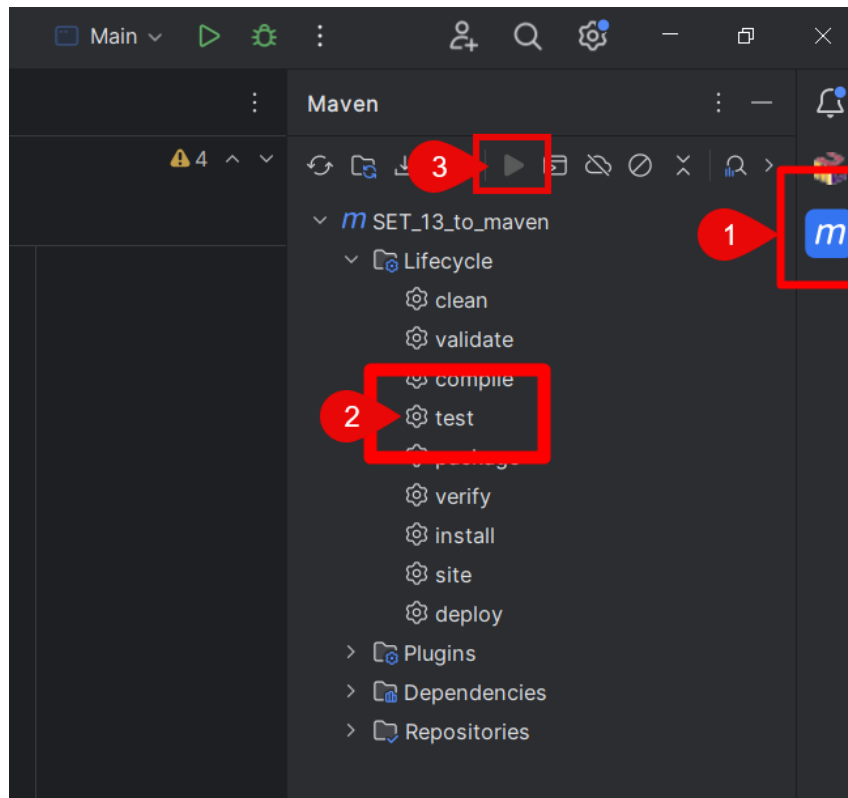
Instruksjon for å kjøre programmet

9.1 Miljøoppsett

For å kunne kjøre koden uten problemer, så må man legge inn maven og Java 21. Noe høyere, hvertfall Java 23 forårsaker at Mockito ikke vil kjøre testene. Deretter er det bare å klonе prosjektet fra Github, eventuelt åpne filene fra mappen i IntelliJ. Maven vil deretter laste alle avhengigheter basert på Pom.xml.

9.2 Kjøre tester

Det er mulig å kjøre alle testene samlet i IntelliJ ved å åpne Maven tool window til høyre, finne mappen Lifecycle og dobbeltklikke på test, som vist i figur 9.1. Maven kjøre alle tester og Surefire plugin vil generere en rapport med detaljer om de utførte testene. De nyeste testresultatene finnes også under GitHub Actions når prosjektet ble senest merget til main branch, som forhåpentligvis er de samme som den lokale testen produserer. **(VIKTIG!)** Denne testen vil ikke fungere om Main.java filen er startet opp, da dette vil føre til konflikter på porter. Kjør derfor denne testen når hovedprogrammet ikke er startet).



Figur 9.1: Kjøre tester i maven

9.3 Kjøre program - GUI

GUI kjøres ved å be IntelliJ kjøre main klassen. En fane i nettleseren skal åpne automatisk så lenge den kjører på port 8080. Sjekk gjerne dette i terminalen. Kan teste MVP sin funksjonalitet ved å gå inn på en av artiklene for å se at produkter fra databasen blir dynamisk lastet inn. Ellers kan en klikke på «Logg inn» knappen øverst til høyre og logge inn med en av de vedlagte detaljene. På samme side er det en knapp for å registrere ny bruker. Prøv å lage en ny bruker, for deretter å logge inn med den nye brukeren sin e-post og passord. Ved å logge inn på john.doe@example.com vil brukerprofilen på venstre side vise brukerprofil detaljer, mens på høyre side hvilke produkter som er registrert på brukeren.

Referanseliste

- [1] ChatGPT, *ChatGPT Share Link*, Accessed: 2024-11-21, 2024. adresse: <https://chatgpt.com/share/673f5968-efa0-8000-98e3-1a724aeb76ef>.
- [2] FasterXML. «Jackson Project Home @github.» (2024), adresse: <https://github.com/FasterXML/jackson>.
- [3] J. IO. «A simple web framework for Java and Kotlin.» (2024), adresse: <https://javalin.io/>.
- [4] A. Isaksen, «Eksamens prosjekt i Webutvikling (ITF10511-1 23H),» [Upublisert eksamens prosjekt], 2023.
- [5] S. Kumar, «A Quick and Practical Example of Hexagonal Architecture in Java,» 2024. adresse: <https://betterprogramming.pub/a-quick-and-practical-example-of-hexagonal-architecture-in-java-8d57c419250d>.
- [6] Digdir. «Universell utforming av ikt.» Accessed: 2024-11-20. (n.d.), adresse: <https://www.digdir.no/standarder/universell-utforming-av-ikt/1499>.
- [7] W. Recommendation. «Web Content Accessibility Guidelines (WCAG) 2.2.» Accessed: 2024-11-20. (okt. 2023), adresse: <https://www.w3.org/TR/WCAG22/>.